# Введение в веб-технологии часть 1

Версия: 2.0
В современном цифровом мире веб-технологии играют ключевую роль в создании, развитии и поддержке информационных систем. Практически каждая организация, будь то малый бизнес или крупная корпорация, использует веб-приложения и сайты для взаимодействия с клиентами, распространения информации и автоматизации бизнес-процессов. Понимание основ веб-технологий становится необходимым не только для разработчиков, но и для специалистов самых разных направлений.

Курс "Введение в веб-технологии" предназначен для формирования базовых знаний о принципах работы Всемирной паутины, структуре веб-приложений, а также языках и инструментах, используемых при создании современных сайтов. В рамках курса вы познакомитесь с такими фундаментальными понятиями, как HTML, CSS, JavaScript, клиент-серверная архитектура, протокол HTTP, и научитесь создавать простые, но функциональные веб-страницы.

Особое внимание в курсе будет уделено вопросам безопасности. Вы узнаете о наиболее распространённых веб-уязвимостях — таких как XSS (межсайтовый скриптинг), Command Execution, а также File Upload. Будут рассмотрены причины их возникновения, а также способы их предотвращения. Демонстрации типичных ошибок при разработке помогут лучше понять, как небезопасный код может поставить под угрозу всю систему.

Цель курса — не только дать слушателям прочную техническую основу, но и сформировать критическое мышление в вопросах безопасности при разработке веб-приложений. Независимо от уровня вашей подготовки, вы получите представление о том, как устроен веб, как создавать безопасные и устойчивые к атакам приложения и как избежать распространённых ошибок.

Добро пожаловать в мир веб-технологий — мир, в котором важно не только создавать, но и защищать.

## Глава 1. Как устроен интернет?

Прежде чем приступить к изучению веб-технологий, важно понять, на чём они основаны. Веб-приложения, сайты и интернет-сервисы работают благодаря инфраструктуре интернета — глобальной сети, которая объединяет миллионы устройств по всему миру. Знание базовых принципов работы интернета поможет вам лучше понимать, как происходит обмен данными, как работают веб-приложения и почему в них возникают уязвимости.

### Что такое интернет?

Слово «интернет» происходит от английского *interconnected networks* — взаимосвязанные сети. Это система, объединяющая множество локальных и глобальных компьютерных сетей в единую структуру. Интернет предоставляет доступ к

разнообразным цифровым ресурсам: веб-сайтам, электронной почте, социальным сетям, видеосервисам, облачным хранилищам и многому другому.

С его помощью люди могут общаться, работать, учиться, развлекаться и обмениваться информацией из любой точки мира.

Интернет называют глобальной сетью. Как и любая сеть, он состоит из узлов — устройств, которые соединены между собой и обмениваются данными.

## Основные типы узлов

В широком смысле к узлам интернета относятся:

- **Коммуникационное оборудование** — маршрутизаторы, коммутаторы, точки доступа и другое «сетевое железо», обеспечивающее передачу данных между устройствами;
- **Клиенты** — устройства, которые запрашивают информацию или услуги;
- **Серверы** — устройства, которые предоставляют информацию или услуги по запросу.

В этом курсе мы не будем подробно рассматривать, как работает коммуникационное оборудование, но важно понимать его роль — оно обеспечивает связь между миллионами устройств, распределённых по всему миру.

## Клиенты и серверы

Рассмотрим подробнее два ключевых типа узлов — клиентов и серверов.

**Клиенты** — это устройства, с которых пользователи получают доступ к интернету. Это могут быть персональные компьютеры, ноутбуки, смартфоны, планшеты, умные телевизоры и другие подключённые к сети устройства. На клиентах работают специальные программы, которые отправляют запросы к серверам. Например, веб-браузер — это программа-клиент, которая запрашивает веб-страницы у веб-сервера.

**Серверы**, в свою очередь, — это устройства (или виртуальные машины), которые обрабатывают запросы клиентов и отправляют им нужную информацию. На сервере работает специальное программное обеспечение — веб-сервер, которое хранит веб-страницы, изображения, скрипты и другие ресурсы, необходимые для отображения сайта.

Процесс взаимодействия клиента и сервера можно описать так:

1. Пользователь вводит адрес сайта в браузере.
2. Браузер (клиент) отправляет запрос на соответствующий сервер.
3. Сервер обрабатывает запрос и отправляет ответ с данными (например, HTML-страницей).
4. Браузер получает ответ и отображает веб-страницу.

Это взаимодействие называется **клиент-серверной архитектурой** и лежит в основе всей веб-разработки.

## Протокол HTTP

Один из ключевых элементов взаимодействия в интернете — это протокол HTTP. Именно он позволяет браузерам и серверам "разговаривать" друг с другом. Понимание его принципов необходимо для создания, тестирования и защиты веб-приложений. Без этих знаний трудно эффективно разрабатывать сайты, отлаживать ошибки и предотвращать уязвимости.

## Как работает протокол HTTP

HTTP (Hypertext Transfer Protocol) — это текстовый протокол, предназначенный для обмена данными между клиентом (например, браузером) и сервером. Каждый обмен включает **запрос от клиента** и **ответ от сервера**. Эти сообщения передаются в виде обычного текста, разбитого на строки и поля.

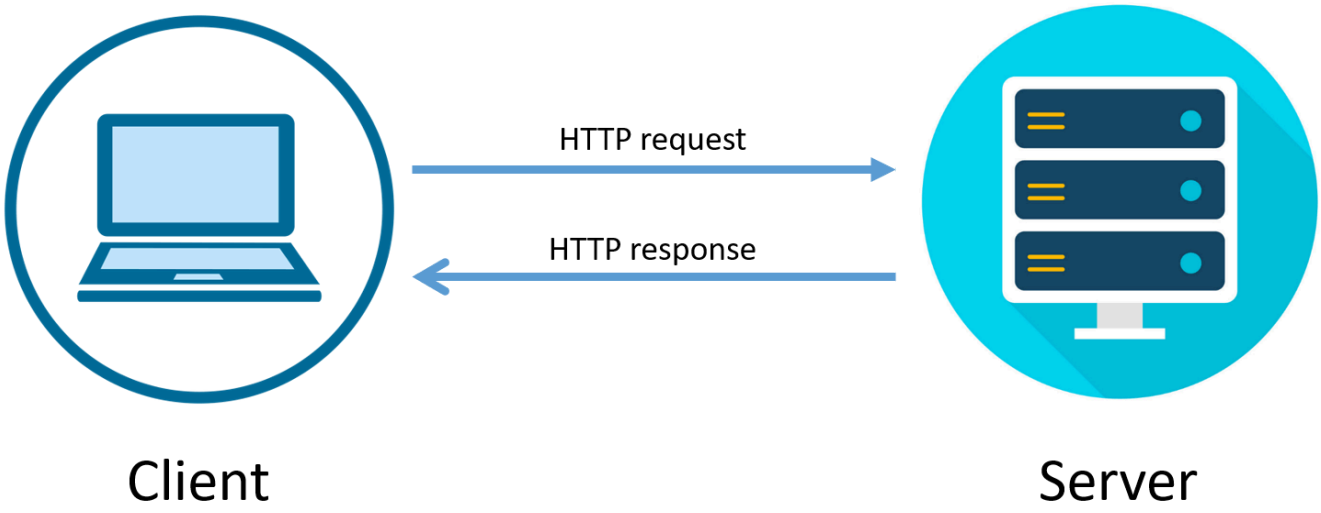



Рисунок 1. Клиент-серверное взаимодействие.

Взаимодействие по протоколу HTTP можно разбить на 4 простых этапа:

1. **Установление соединения**
   Когда клиент хочет получить данные с сервера, например просмотреть содержимое главной страницы сайта, он вводит адрес сайта в адресной строке браузера. Веб-браузер, в свою очередь отправляет HTTP-запрос (HTTP-request) на сервер.
   На рисунке изображены составляющие HTTP-запроса.




Рис. 2 HTTP-запрос и его составляющие.

HTTP-запрос включает в себя:

- заголовки:
- HTTP-метод (например, GET, POST или PUT),
- версию протокола (protocol version)
- URL ресурса;
- другие заголовки, которые могут содержать дополнительную информацию;
- тело запроса (опционально), используется, например, при отправке данных форм.

2. **Обработка запроса на сервере**
   Когда сервер получает HTTP-запрос, он анализирует его, выполняет запрошенное действие, (например, возвращает веб-страницу или обрабатывает данные формы) и формирует HTTP-ответ.
   Обратим внимание, что содержимое возвращаемой сервером веб-страницы может как зависеть, так и не зависеть от параметров HTTP-запроса.
   Если содержимое страницы все-таки зависит от параметров запроса, то такая страница обычно формируется с помощью серверного языка программирования, PHP, который мы рассмотрим позже в данном курсе. Ответ сервера, который зависит от параметров запроса называется динамическим.
3. **Отправка ответа клиенту**
   После того как сервер обработал запрос, он отправляет HTTP-ответ обратно клиенту.

“МГТУ им. Н.Э. Баумана/ЦК/Для курса по ИБ/1_Введение в веб-технологоии/картинки/http-response.jpg” не может быть найдена.

Рис. 3. HTTP-ответ

HTTP-ответ включает:

- заголовки
  - версию протокола;
  - статус ответа (например, 200 OK, 404 Not Found);
  - заголовки ответа, содержащие дополнительные данные (например, тип содержимого, дата и время).
- содержимое, называемое телом ответа. (например, HTML-код веб-страницы, JSON-данные или другой контент, который браузер должен отобразить.).

4. **Закрытие соединения**
   После завершения передачи данных сервер закрывает соединение, если это необходимо.

Рассмотрим подробнее некоторые упомянутые выше понятия.

## HTTP-методы

Одной из составляющих HTTP-запроса является HTTP-метод.
HTTP-методы - это специальные команды, которые используются в протоколе HTTP для определения типа операции, которую клиент (например, браузер) хочет выполнить на сервере.
Каждый HTTP-метод представляет собой определенное действие, которое должен выполнить сервер в ответ на запрос.

Перечислим наиболее распространенные HTTP-методы и дадим им краткие характеристики.

- GET - метод для чтения данных с сайта. Например, для доступа к указанной странице. Он говорит серверу, что клиент хочет прочитать указанный документ. При использовании GET-запрса информация передается прямо в HTTP-заголовке.

  Пример GET-запроса. Информация передаётся прямо в заголовке.

  ```
  GET /blog/?name1=value1&name2=value2 HTTP/1.1
  Host: example.com
  ```

- POST - метод для отправки данных на сайт. Чаще всего с помощью метода POST передаются данные форм. При использовании POST-запроса. информация передается в теле запроса.

Пример POST-запроса. Информация передаётся в теле запроса:

```
POST /blog/ HTTP/1.1
Host: example.com
name1=value1&name2=value2
```

- PUT - метод для создания или обновления ресурса на сервере. Например, вы написали пост, а потом решили его полностью изменить. Для этого нужно подготовить актуальный текст и сделать запрос `PUT` с новым постом, который полностью заменит предыдущий.
- DELETE - метод для удаления ресурса на сервере.
- HEAD - аналогичен методу GET, но сервер отправляет в ответ только заголовки ответа без тела сообщения.

Эти методы позволяют клиентам и серверам взаимодействовать друг с другом, определяя тип операции, которую нужно выполнить с ресурсом. Каждый метод имеет свое предназначение и правила использования в рамках протокола HTTP.

## Адресация в интернете: URL и доменные имена

### Что такое URL?

Одной из составляющих HTTP-запроса является URL.

URL (Uniform Resource Locator) - это адрес ресурса в сети интернет. URL позволяет пользователям и программам обращаться к различным ресурсам в глобальной сети, указывая точное местоположение и способ доступа к ним.
Именно URL мы вводим в адресную строку браузера.
Рассмотрим его строение на простом примере:




Рис. 4. URL

URL обычно состоит из четырех частей:

- **протокола**, протокол, по которому мы обращаемся к сайту. Обычно это HTTP или HTTPS;
- **доменного имени сервера**, указывающего, какой именно веб-сервер из глобальной сети должен быть запрошен. В качестве альтернативы может быть использован и IP-адрес (например, http://93.184.216.34), но это делается редко, поскольку запоминать IP сложнее;

- **порта**. Порт является техническим параметром, используемый для доступа к ресурсам на веб-сервере. Зачастую подразумевается, что веб-сервер использует стандартные порты HTTP-протокола (80 для HTTP и 443 для HTTPS) для доступа к своим ресурсам и в таком случае, порт в URL не указывается:
- **пути до файла на веб-сервере.** В прошлом, адрес отображал местоположение реального файла в реальной директории на веб-сервере. В наши дни это чаще всего абстракция, позволяющая обрабатывать адреса и отображать тот или иной контент.

На практике, URL — это способ точно указать, какой ресурс в интернете нужно получить и как.

### Доменное имя

Для удобства пользователей вместо числовых IP-адресов используются доменные имена. Это текстовое имя, которое соответствует IP-адресу конкретного сервера. Например, доменное имя example.com соответствует IP-адресу 93.184.216.34.

Когда пользователь вводит доменное имя в браузере, браузер отправляет запрос на DNS-сервер, чтобы получить IP-адрес, соответствующий доменному имени.

Затем браузер отправляет запрос на сервер по полученному IP-адресу и получает HTML-страницу, которую отображает для пользователя.

Таким образом, взаимодействие со страницей на сервере в реальности включает в себя следующие шаги (Рис. 5):

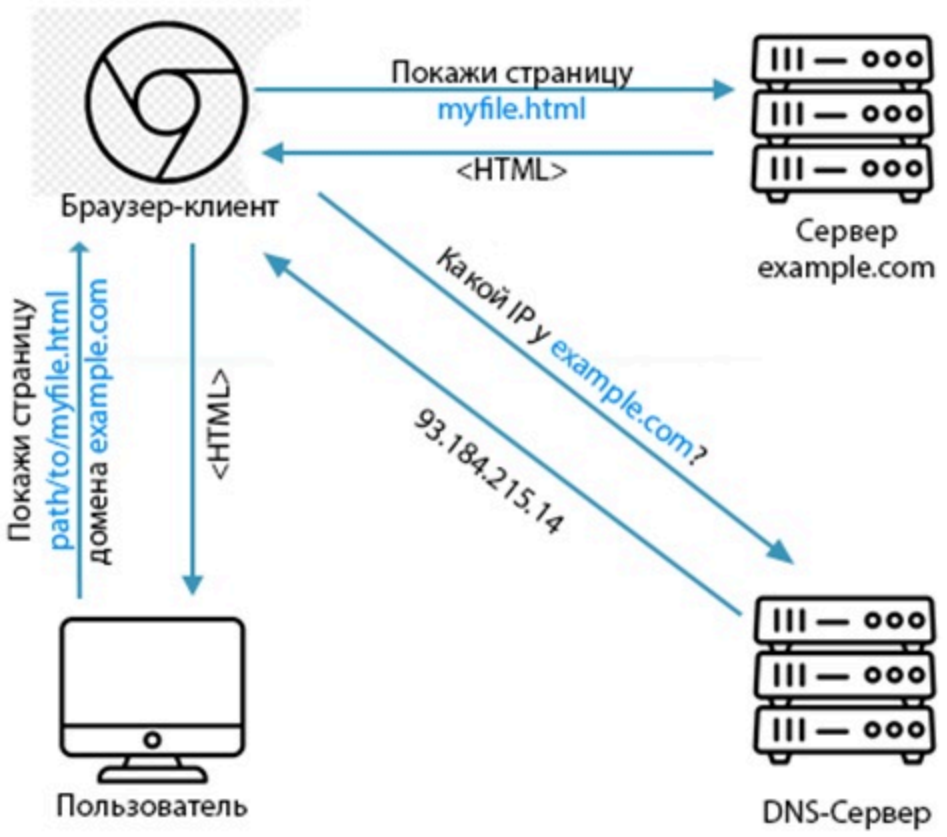



Рис. 5. DNS

1. Пользователь вводит доменное имя в браузере;
2. Браузер отправляет запрос на DNS-сервер, чтобы получить IP-адрес, соответствующий доменному имени;
3. Браузер отправляет запрос на сервер по полученному IP-адресу.
4. Сервер обрабатывает запрос и отправляет обратно HTML-страницу.
5. Браузер отображает HTML-страницу для пользователя.

## HTTP статусы ответов

Одной из составляющих HTTP-ответа является HTTP статусы ответов, также называемые кодами состояния.

HTTP статусы ответов - это числовые коды, которые отправляются сервером в ответ на запрос клиента, чтобы сообщить о результате выполнения запроса.
Каждый HTTP статус состоит из трех цифр и имеет свою специфическую интерпретацию.

Перечислим некоторые из наиболее распространенных HTTP статусов:

- 200 OK - успешный запрос, сервер успешно обработал запрос клиента.
- 404 Not Found - запрашиваемый ресурс не найден на сервере.
- 500 Internal Server Error - внутренняя ошибка сервера, сервер не смог обработать запрос из-за ошибки.
- 301 Moved Permanently - запрашиваемый ресурс был перемещен на другой URL постоянно.
- 403 Forbidden - доступ к запрашиваемому ресурсу запрещен.
- 302 Found - запрашиваемый ресурс временно перемещен на другой URL.

HTTP статусы ответов помогают клиентам понять результат выполнения их запросов и принять соответствующие действия в зависимости от полученного статуса.

## Как устроены веб-сайты

После знакомства с протоколом HTTP давайте разберёмся, из каких компонентов состоит современный веб-сайт и какие технологии обеспечивают его работу.

### Две стороны веб-сайта

Каждый веб-сайт состоит из двух основных частей — **клиентской** и **серверной**. Они работают вместе, но выполняют разные задачи.

**Клиентская часть**, также называемая *frontend*, отвечает за то, что пользователь видит на экране и как он взаимодействует с сайтом. Это структура страницы, оформление, кнопки, меню, анимации, всплывающие окна и другое. Все эти элементы работают прямо в браузере пользователя, без необходимости постоянно обращаться к серверу.
Основные технологии клиентской части — это HTML, CSS и JavaScript.

**Серверная часть**, или *backend*, работает "за кулисами" — на удалённом сервере. Она обрабатывает запросы от браузера, взаимодействует с базой данных, выполняет бизнес-логику приложения и формирует ответы, которые отправляются обратно клиенту.
На серверной стороне используются такие технологии, как PHP, базы данных (например, MySQL) и язык запросов SQL.

Запомним, что клиентская часть отвечает за внешний вид и поведение сайта, а серверная — за обработку данных и «интеллект» сайта.

## HTML

HTML (Hypertext Markup Language) — это стандартный язык, который используется для разметки веб-страниц. Представьте, что ваша веб-страница — это книга. Тогда HTML — это скелет этой книги, костяк, который определяет структуру: где будет заголовок, абзацы, изображения и так далее. HTML - это клиентская веб-технология.

## CSS

CSS (Cascading Style Sheets) — CSS используется для стилизации веб-страниц. Он детально определяет внешний вид каждого элемента HTML: какой у элемента будет размер, цвет, где на странице он будет расположен, какой шрифт будет использоваться и т. д. CSS - это клиентская веб-технология.

## Javascript

JavaScript является языком программирования, который используется для добавления интерактивности на веб-сайты. Он позволяет создавать динамические элементы, анимацию, обработку событий и взаимодействие с пользователем.
Сценарии JavaScript выполняются в браузере пользователя, поэтому JavaScript - это клиентская веб-технология.

## PHP

PHP - это скриптовый язык программирования, который выполняется на сервере и генерирует HTML код, который затем отправляется клиентскому браузеру.
Программа на языке PHP называется сценарием. Сценарий находится на сервере и запускается в ответ на запрос от браузера.
Основная задача PHP — это «оживление» HTML страниц. Обычные HTML-страницы статичны. Это означает, что после того как страницу создали и загрузили на сайт, при каждом обращении к этой странице браузер будет показывать её каждому пользователю в неизменном виде.
Однако практически всегда пользователь приходит на сайт за актуальным состоянием какой-то информации, которая постоянно меняется.
Примерами такой информации может быть курс валют на сегодня или прогноз погоды на завтра.
Если использовать только HTML, то решить задачу получения актуальной информации не получится. И тут на помощь приходит PHP. Он примет входящий запрос от веб-

сервера, выполнит php-сценарий и вернет веб-серверу результат в виде готового HTML-кода. Сервер отправляет этот результат в браузер, который, в свою очередь, отображает её пользователю. После этого видно свежий курс валют или погоду на желаемый день.

### Базы данных и SQL

Еще одним примером серверной веб-технологии служит база данных.

База данных (БД) - это организованная коллекция данных, которая хранится и управляется с помощью специальных программных средств - систем управления базами данных (СУБД). Базы данных используются для хранения различной информации, такой как текст, числа, изображения, видео и другие типы данных, и предоставляют возможность эффективного доступа, обновления и анализа этих данных.

СУБД — это отдельная программа, которая работает как сервер.

Для хранения и управления данными, используемыми на веб-сайте, часто применяются реляционные базы данных.

Реляционная база данных (РБД) - это тип базы данных, который организует данные в виде таблиц, состоящих из строк и столбцов. В реляционных базах данных данные представлены в виде набора связанных таблиц, где каждая таблица представляет собой отдельную сущность или отношение.

В данном курсе мы будем использовать реляционную базу данных под названием MySQL.

Создавать свои базы данных, таблицы, наполнять их данными и получать данные из них можно прямо из PHP-сценариев, а также используя СУБД, однако для выполнения этих операций прежде придётся познакомиться с ещё одним языком программирования — SQL.

SQL или Structured Query Language (язык структурированных запросов) — язык программирования, предназначенный для управления данными в БД.

На языке SQL выражаются все действия, которые можно провести с данными: от записи и чтения данных, до администрирования самой СУБД.

Одним из "золотых" принципов МГТУ им Н.Э. Баумана является обучение через практику. Поэтому закрепим полученные знания на примере создания маленького, но гордого веб-сайта.

## Переходим к практике

Вы шаг за шагом создадите простой веб-сайт, использующий HTML, стилизованный с помощью CSS, с интерактивными элементами на JavaScript, обработкой данных через PHP и сохранением информации в базе данных MySQL.

### Знакомство с HTML. Основы разметки.

Веб-страницы являются основой для представления информации в сети Интернет.

Информация на веб-страницах находится в структурированном виде.

Зададимся вопросом, а зачем, собственно, структурировать информацию?

Дело в том, что структурирование информации играет важную роль в обеспечении её понимания и удобства использования. Вот несколько причин, почему структурирование информации важно:

1. Улучшение понимания: Структурированная информация обычно более легко усваивается и понимается. Путем организации данных в логические категории и подкатегории мы создаем более ясное представление о том, как информация связана между собой.
2. Облегчение навигации: Структурирование информации помогает пользователям быстро находить нужные им данные. Например, веб-сайты с хорошо спроектированными структурами позволяют пользователям легко перемещаться между различными разделами и страницами.
3. Упрощение анализа: Когда информация структурирована, проще проводить анализ и извлекать полезные знания. Например, базы данных с четкой структурой позволяют пользователям быстро находить нужные данные и проводить анализ.
4. Повышение эффективности: Структурированная информация способствует повышению эффективности работы, поскольку пользователи тратят меньше времени на поиск нужной информации.
5. Улучшение визуального восприятия: Хорошо структурированная информация обычно выглядит более привлекательно и профессионально, что повышает ее визуальное восприятие.

Язык разметки HTML определяет структуру веб-страниц за счет добавления к информации различных типов специальных управляющих структур, называемых тэгами.

Наш веб-браузер умеет обрабатывать HTML-тэги и представлять обрамленную ими информацию в красивом структурированном виде.

Так, HTML разметка позволяет определить структуру веб-страницы, такую как заголовки, параграфы, списки, таблицы и так далее.

Отметим, что HTML-теги имеют и семантическое значение. Это означает, что они сообщают браузеру и поисковым системам о значении содержимого. Например, тег `<h1>` обозначает заголовок первого уровня, а тег `<p>` - параграф. Это помогает улучшить доступность и SEO-оптимизацию веб-страниц.

## Создание первой веб-страницы

Создайте на ваших компьютерах в удобном месте директорию с названием BMSTU_website. В ней мы будем размещать файлы создаваемого сайта.

Создадим нашу первую веб-страницу.

Для этого необходимо:

- создать файл с расширением index.html
- наполнить его текстовой информацией
- структурировать текстовую информацию, заключив её в соответствующие тэги

Сделаем это.

Создавать сайт мы будем с импользованием бесплатного редактора кода Visual Studio Code (Далее - VSC).

### Установка редактора Visual Studio Code

Чтобы установить редактор, переходим на сайт https://code.visualstudio.com/ и нажимаем большую кнопку с надписью "Скачать" (Download)

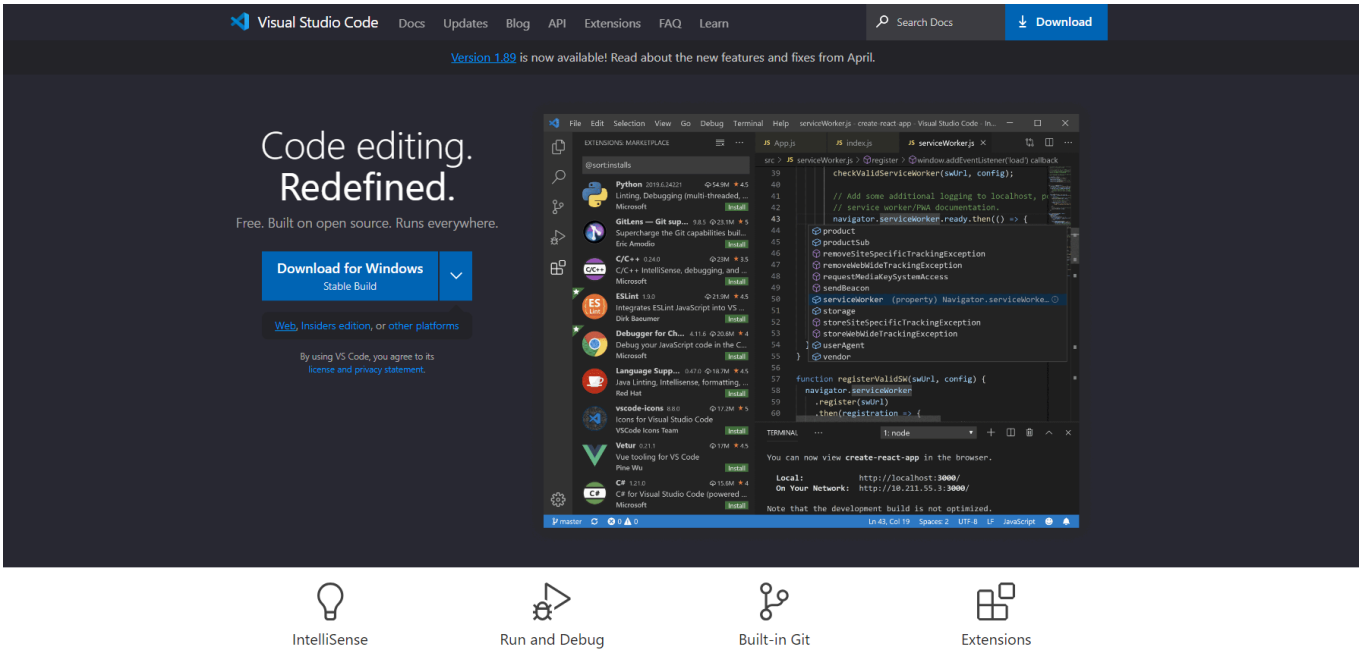



Рис. 6. Главная страница сайта VSCode

Версия, предназначенная для вашей операционной системы будет скачана на компьютер. Далее, перейдите в папку "Загрузки" и дважды кликните на скачанный файл и следуйте указаниям установщика.

Также вы можете воспользоваться веб-версией описываемого редактора, расположенной по адресу https://vscode.dev/

### Создание файла с расширением .html

Откройте VSC, в левом верхнем углу нажмите на пункт меню "Файл" (File), в выпадающем меню выберите пункт "Открыть папку" (Open Folder).
После этого в проводнике укажите путь до предварительно созданной вами директории под названием BMSTU_website.

После открытия директории нажмите на значок создания нового файла (на рисунке ниже)

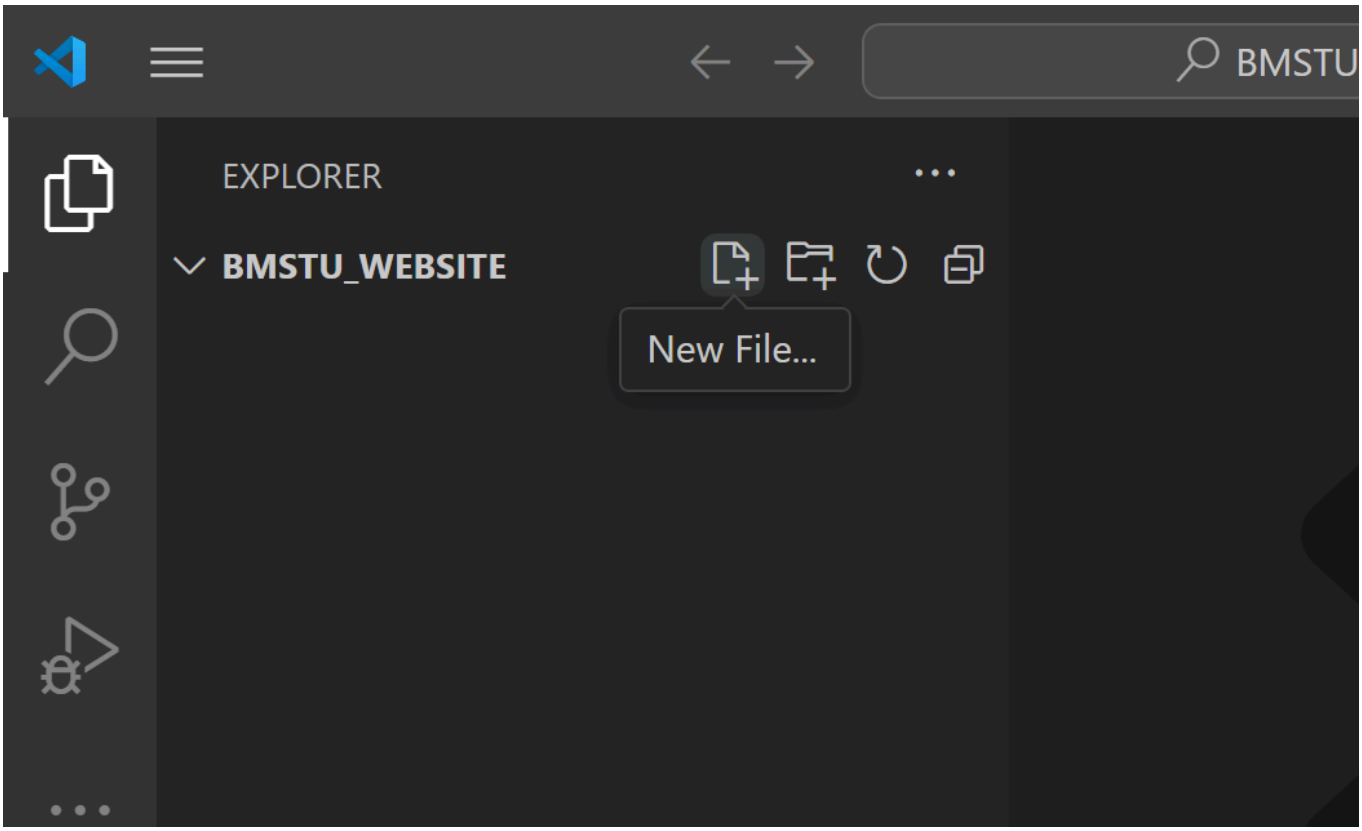



Рис.7. Создание нового файла в VSC

Далее укажите название и расширение файла: index.html (Рис. 8)

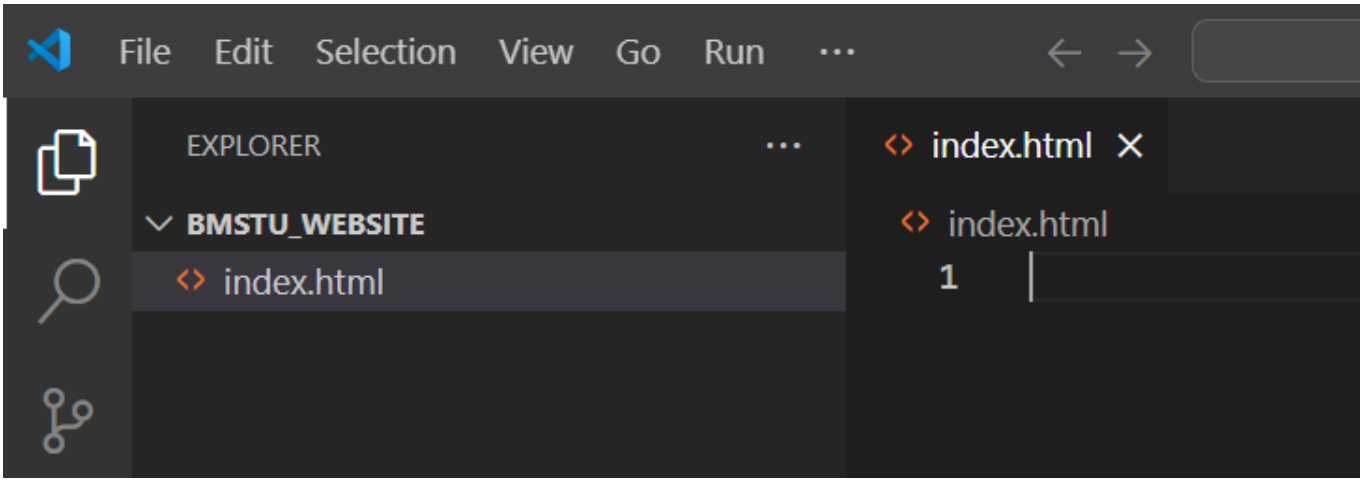



Рис.8. Создание нового файла в VSC. Продолжение

Вы великолепны. Приступаем к написанию кода.

**Создаем главную страницу**

Первый файл, с которого начинается любой веб-сайт, обычно называется `index.html`. Это главная страница сайта — именно её по умолчанию открывает браузер, когда пользователь переходит по адресу сайта (например, `https://example.com`).

Создавая файл `index.html`, мы начнём практическое знакомство с языком разметки **HTML**.

Язык HTML состоит из специальных элементов — **тегов**. Каждый тег сообщает браузеру, *что* находится внутри него: заголовок, абзац, изображение, ссылка, таблица и т. д.

Тег пишется в угловых скобках, например: `<p>` — абзац, `<h1>` — заголовок первого уровня, `<img>` — изображение.

Тэги бывают двух типов:

- парные тэги, состоящие из открывающего и закрывающего тега.
  Например:

  ```
  <p>Это абзац текста</p>
  ```

  Здесь `<p>` — открывающий тег, `</p>` — закрывающий тег. Всё, что между ними, — это содержимое.
- непарные, которые испоьзуются самостоятельно, без закрывающей пары.
  Примеры: `<img>`, `<br>`, `<hr>`

**С чего начинается HTML-документ?**

Каждый HTML-документ **обязательно** должен начинаться с декларации типа документа — `<!DOCTYPE html>`. Это называется **доктайп**.

Он сообщает браузеру, какую версию HTML использовать для правильного отображения страницы.
Раньше он выглядел так:

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN"
  "http://www.w3.org/TR/html4/strict.dtd">
```

Мы будем использовать современный стандарт **HTML5**, и его доктайп выглядит так:

```
<!DOCTYPE html>
```

Добавьте строку декларации типа документа в файл index.html

Продолжаем создание страницы!

**Скелет страницы**

Поздравляю — мы почти у цели. Сейчас вы сделаете то, что делают миллионы веб-разработчиков каждый день: создадите свою первую настоящую страницу.

Для начала — важное правило: каждая HTML-страница должна быть правильно "собрана". Как книга состоит из обложки, оглавления и глав, так и веб-страница — из своих обязательных элементов. Без них браузер запутается и не поймёт, что вы вообще от него хотите.

Любая страница — будь то сайт банка или личный блог про котиков — состоит из как минимум трёх важных частей: `<html>`, `<head>` и `<body>`.

Тэг `<html>` — самый главный. Он располагается в документе сразу после доктайпа и содержит все остальные тэги, включая `<head>` и `<body>`. Тэгу `<html>` обычно добавляют важный атрибут `lang` (сокращение от английского «language»), в котором задаётся язык документа. Наш сайт будет русскоязычным, поэтому в `lang` пропишем такое значение:

```
<html lang="ru">
```

Тег `<head>` — «голова» страницы. Здесь спрятаны её мысли: служебная информация, заголовок, настройки, ссылки на стили и скрипты. Пользователь этого не видит, но браузеру это жизненно важно.

Тэг `<body>` — «тело» страницы. Всё, что вы видите на экране — заголовки, абзацы, изображения — находится внутри этого тега.

Все тексты и картинки мы будем добавлять внутрь тэга `<body>`. В документе тэги `head` и `body` могут быть использованы только один раз.

Внутри тэга `<body>` расположим тэг `<main>`. Он выделяет основное содержание страницы, которое не повторяется на других страницах.

Добавьте в index.html необходимые тэги.

В файле `текст_для_сайта.txt` размещен текст про МГТУ. Перенесите этот текст внутрь тэга `<body>`

```
<!DOCTYPE html>
<html lang="ru">
  <head>
  </head>
  <body>
  <main>
    Инженеры создают будущее - мы создаём инженеров
    Московский государственный технический университет им. Н. Э. Баумана — российский национальный исследовательский университет, научный центр и особо ценный объект культурного наследия народов России.
```

```
    Факты об университете
    190+ лет истории
    9 000+ сотрудников, из них 25% — моложе 39 лет
    19 факультетов
    31 000+ студентов и аспирантов, из них 30% — специалитет

    Наша миссия
    Мы – высшее техническое училище. Наша миссия – создавать научно-
технологическое будущее, воспитывать новые поколения русских инженеров. Мы
воспитываем инженеров и технологических лидеров, связываем поколения,
укрепляем обороноспособность и технологическую независимость России, служим
источником идей, обеспечиваем экспертизу и участвуем в разработке и
внедрении новой техники и технологий для устойчивого будущего.

    Русский метод обучения
    Основной принцип бауманской инженерной школы — сочетание глубоких
теоретических знаний и обширных практических навыков.
    Университет готовит специалистов, способных вести разработки в сфере
высоких технологий, наукоемких производств, владеющих знаниями в области
экономики, менеджмента, цифровых решений, предпринимательства.
    Программы обучения построены в партнерстве с ведущими компаниями и
предприятиями России.
</main>
  </body>
</html>
```

Откройте получившейся файл в браузере (Рис. 9).

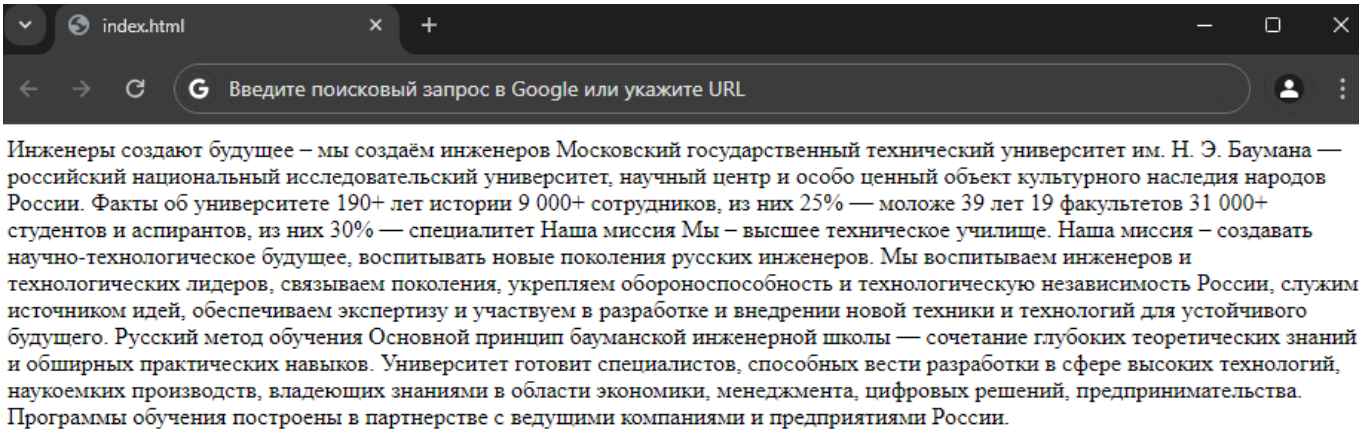



Рис. 9. Неразмеченный текст будущего сайта в окне браузера.

В браузере отобразится ваш первый веб-сайт. По сути - это настоящий веб-сайт, только некрасивый. Если файл index.html поместить на реальный сервер, то запрос к серверу будет возвращать точно такую же веб-страницу.
Как вы видите, текст в окне браузера отображается одним "куском". Так происходит потому что текст внутри тега `<body>` не размечен.

На данный момент разместив текст внутри тега `<body>` мы сказали браузеру, какой контент будет располагаться внутри его главного окна окна браузера.

### Тэг head

Тег `<head>` — это специальный раздел HTML-документа, в котором хранится **служебная информация** о странице. Хотя его содержимое не отображается напрямую в окне браузера, именно здесь задаются важнейшие параметры страницы: название, кодировка, подключение шрифтов, стилей и скриптов, ключевые слова и описание для поисковых систем.

Тег `<head>` всегда размещается **внутри** тега `<html>` и **до** тега `<body>`. Его можно представить как «невидимую часть айсберга» — пользователь её не видит, но без неё страница не будет работать корректно.

### Если вместо букв вы вдруг видите "кракозябры"

Обычно этого не происходит, этот раздел включен на всякий случай.

Это (появление "кракозябр" вместо красивых букв русского алфавита) может быть вызвано тем, что веб-сервер использует кодировку по умолчанию, и если она не указана явно, то на страницах могут отображаться искажённые символы
Добавьте следующую строку внутрь тэга `<head>`:

```
<meta charset="UTF-8"/>
```

### Тэг title

Одним из важнейших элементов внутри `<head>` является тег `<title>`. Именно он определяет **название страницы**, которое вы видите во вкладке браузера.

Пример использования `<title>` в коде:

```
<head>
  <title>Сайт о МГТУ им. Н.Э. Баумана</title>
</head>
```

Текст в `<title>` должен описывать содержимое документа в целом. По заголовку должно быть понятно, о чём эта страница, даже когда она не открыта в браузере, а отображается в результатах поиска или в браузерных закладках.
Кроме того, название страницы используется в результатах поиска (например, в Яндексе или Google).

Добавьте тег `<title>` в index.html и посмотрите что получится (Рис. 10)

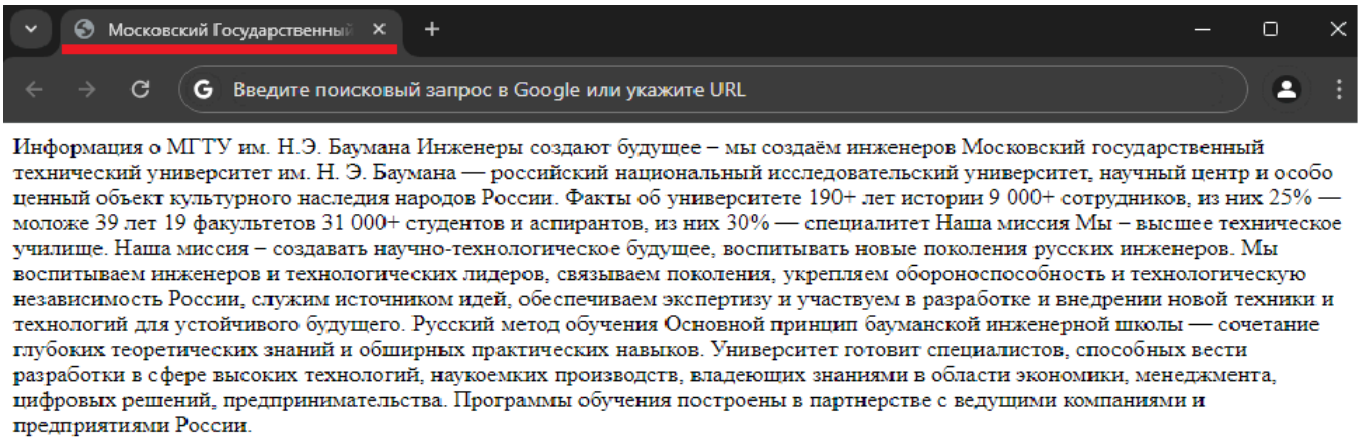



Рис. 10. Появление тэга `<title>` в index.html

### Комментарий в HTML-коде

Комментарий в HTML-коде задаётся так:

```
<!— любой текст —>
```

Текст внутри комментария не отображается браузером на странице. Комментарии обычно используются в следующих случаях:

- Для комментирования кода. Всегда полезно оставить подсказку.
- Для временного отключения кода. Удалять код неудобно, так как его надо будет восстанавливать, а закомментировать и потом раскомментировать — самое лучшее решение.

Комментарии можно использовать в любом месте страницы, кроме тега `<title>` — внутри него они не работают.

### Тэг link

С помощью непарного тега `<link>` к странице подключаются CSS-файлы стилей. Создадим такой файл и подключим его к нашей странице.

Здесь начинается ваше знакомство с **CSS** — языком, который управляет внешним видом веб-страниц. Если HTML — это каркас, структура и содержание, то CSS — это оформление: цвета, шрифты, отступы, фон, размеры и расположение элементов.

Эти два языка работают **в связке**: HTML говорит браузеру, *что нужно отобразить*, а CSS — *как это должно выглядеть*. Без HTML на странице просто нечего было бы показывать, а без CSS — всё выглядело бы одинаково и скучно. Только вместе они превращают код в настоящую, красиво оформленную веб-страницу.

Уточним, что ни HTML, ни CSS **не являются языками программирования**.

**HTML** — это язык разметки, он описывает, *что должно быть на странице* (заголовки, абзацы, изображения и т. д.).

**CSS** — это язык стилей, который определяет, *как всё это должно выглядеть* (цвета, шрифты, отступы и т. п.).

В папке BMSTU_WEBSITE создайте файл `style.css`
Заполните его данным содержимым:

```
main {
    background-color: lightblue;
    font-size: 16px;
}
```

После этого подключите данный файл стилей в тэг `<head>` следующим образом:

```
<head>
  <link href="style.css" rel="stylesheet">
    <title>Московский Государственный Технический Университет им. Н.Э. Баумана</title>
</head>
```

**Атрибуты тегов — дополнительные настройки**

Помимо названия, каждый HTML-тег может содержать **атрибуты** — это дополнительные параметры, которые уточняют, как должен работать или отображаться элемент. Атрибуты всегда указываются внутри открывающего тега и состоят из пары «имя = значение».

Рассмотрим детальнее тег link с атрибутами:

```
<link href="style.css" rel="stylesheet">
```

У `<link>` в атрибуте `href` задаётся адрес стилевого файла,
а атрибут `rel` со значением `stylesheet` говорит браузеру, что мы подключаем именно стили, а не что-то другое.

Сохраните файл, обновите страницу в браузере — и вы заметите, что **цвет фона и размер шрифта поменялись**. Это значит, что CSS начал работать и управлять внешним видом вашего HTML-документа.

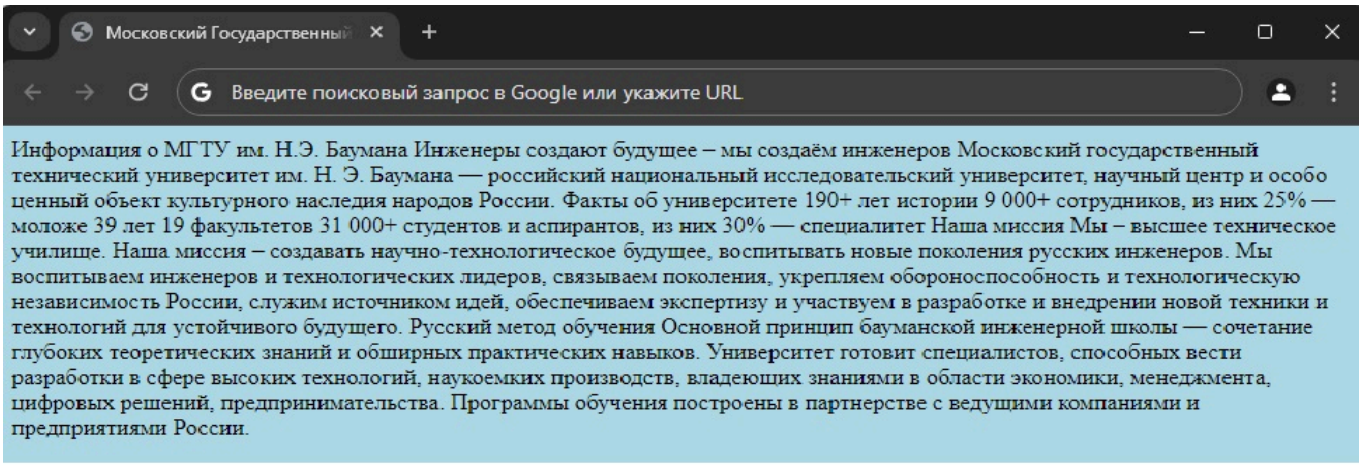



Рис. 11. Файл стилей `style.css` применен к файлу в index.html

Всё, что мы написали в файле `style.css`, называется **правилом CSS**. Каждое такое правило определяет, как должен выглядеть определённый элемент на странице.

CSS-правило состоит из **селектора** и **набора объявлений** (иногда их называют инструкциями), которые заключены в фигурные скобки. Каждое объявление внутри фигурных скобок — это пара **«свойство: значение»**, и в конце ставится точка с запятой `;` (Рис. 12).




Рис. 12. Анатомия правила CSS

**Селектор** — это указатель, к каким HTML-элементам нужно применить заданные стили. Он как бы "выбирает" те элементы на странице, к которым CSS должен примениться. Например, в таком правиле:

```
body {
    background-color: lightblue;
    font-size: 16px;
}
```

Селектор `body` указывает, что стили относятся ко **всему тегу** `<body>` — то есть ко всей странице.

В нашем случае, мы привязали к тэгу `<body>` два стилевых свойства:
Первое, `background-color: lightblue;` означающее, что мы задаем для данного тэга светло-голубой фон (фон цвета lightblue)

Второе, `font-size: 16px;` означающее, что внутри этого тэга размер шрифта, которым написан текст составит 16 пикселей.

В приведённом примере мы использовали **селектор типа** — он выбирает все элементы по названию тега (`body`, `p`, `h1` и т. д.).

Однако существует и множество других типов селекторов:

- по классу (`.название`),
- по идентификатору (`#название`),
- по вложенности (`article p`),
- по состоянию (например, `a:hover` — ссылка при наведении).

Подробнее о разных типах селекторов можно узнать на официальном сайте MDN Web Docs: https://developer.mozilla.org/ru/docs/Learn/CSS/Building_blocks/Selectors

**Шапка и подвал**

Кроме уникального основного содержимого страницы, описанного внутри тэга `<main>`. у нас есть повторяющиеся на других страницах вводная часть и заключительная часть.

Вводную часть страницы, которую чаще называют «шапкой», описывает тег `<header>`.

Аналогично ему, заключительную часть страницы, или «подвал», описывает тег `<footer>`.

Добавим данные тэги в код нашей страницы.

```
<body>
  <header>
        Шапка сайта. Здесь будет навигация и логотип.
  </header>
  <main>
  ...
  </main>
  <footer>
    Подвал сайта.
  </footer>
</body>
```

Теперь придадим нашим новым разделам внешний вид. Для этого добавим в файл `style.css` стили для шапки (`header`), подвала (`footer`) и доработаем стили для основного содержимого (`main`)::

```
header {
    background-color: azure;
```

```
    font-size: 18px;
    border-bottom: 2px solid grey;
    height: 10vh;
}

main {
    background-color: lightblue;
    font-size: 16px;
    height: 80vh;
}

footer {
    background-color: azure;
    font-size: 18px;
    border-top: 2px solid grey;
    height: 10vh;
    padding: 10px;
}
```

Чтобы глубже понять, как именно работают отступы, границы и размеры в CSS, нам нужно познакомиться с важной концепцией — **блочной моделью CSS**. Именно она отвечает за то, как браузер "рисует" каждый элемент на странице.

Об этом — в следующем разделе.

**Блочная модель CSS**

Чтобы правильно отображать элементы на странице, браузер должен понимать, **как именно их “рисовать”**. Для этого в CSS существует ключевая концепция — **блочная модель** (Box Model). Она определяет, как элементы HTML будут отображаться в браузере, учитывая их размеры, отступы, границы и заполнение. Блочная модель изображена на рисунке 13.

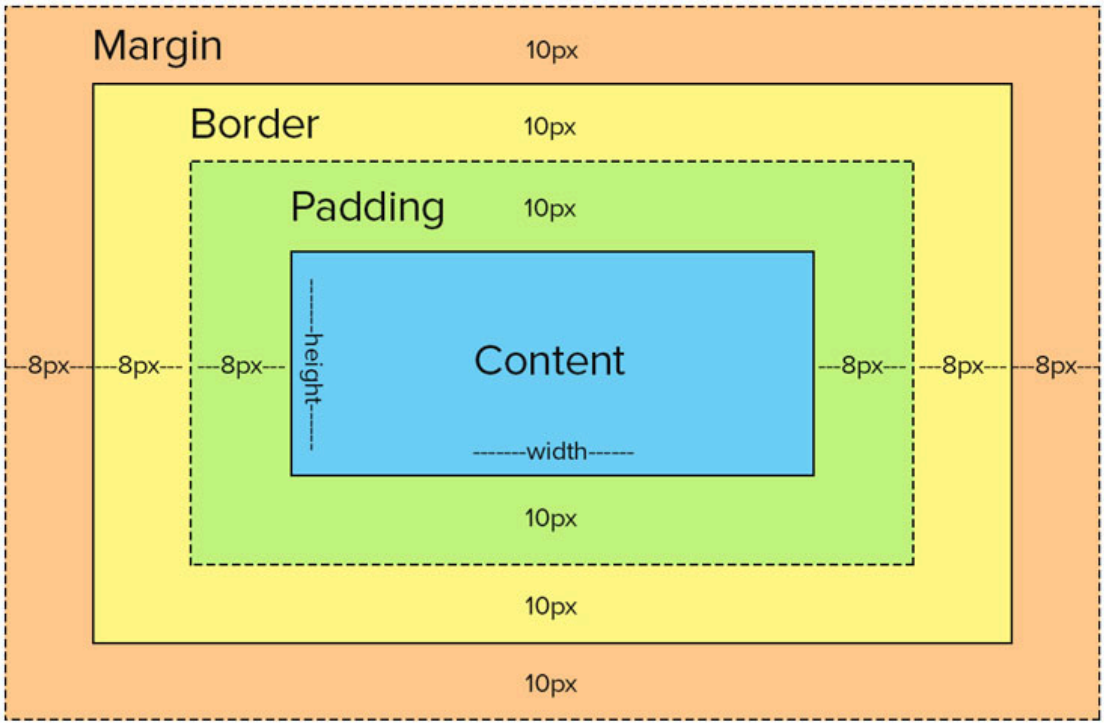



Рис. 13. Блочная модель CSS.

Объясним данное понятие на простой аналогии.
Представим, что каждый элемент HTML на веб-странице - это коробка. Эта коробка имеет внутреннее содержимое: текст, изображения и т.д. (content), отступы вокруг содержимого (padding), рамку вокруг себя (border) и внешние отступы вокруг себя (margin).

1. Содержимое (Content): Это то, что находится внутри коробки, например, текст, изображение или другие элементы. Именно это мы хотим показать пользователю.
2. Внутренние отступы (Padding): Это пространство между содержимым и границей коробки. Например, если у тебя есть текст внутри коробки, то отступы позволяют создать небольшое расстояние между текстом и границей. Padding как бы «отталкивает» текст от краёв, создавая свободное пространство внутри.
3. Рамка (Border): Это линия или рамка вокруг коробки. Она может быть тонкой или толстой, цветной или прозрачной — всё зависит от ваших настроек в CSS.
4. Внешние отступы (Margin): Пространство между текущей коробкой и другими коробками на странице. Это пространство вокруг самой коробки, которое отделяет её от других элементов на странице. Margin используется, чтобы элементы не слипались, а красиво располагались относительно друг друга.

Знание блочной модели даёт полный контроль над расположением и оформлением элементов. Используя блочную модель мы можем легко стилизовать элементы на веб-странице.

**Шапка и подвал. Продолжение**

Вооружившись новыми знаниями, опишем назначение новых правил CSS:

- `border-bottom: 2px solid grey;` данное правило рисует сплошную (solid) серую (grey) рамку (border) снизу "коробки" (bottom) шириной 2 пикселя (2px)
- `border-top: 2px solid grey;` данное правило рисует сплошную (solid) серую (grey) рамку (border) сверху "коробки" (top) шириной 2 пикселя (2px)
- `height: 10vh;` данное правило описывает высоту элемента. `vh` - это относительная единица измерения. 1 vh эквивалентно 1% высоты окна браузера.
- `padding: 10px;` данное правило задает внутренний отступ между содержимым тэга (в данном случае теэга `footer`)

Обновим страницу и посмотрим что получилось (Рис. 14)

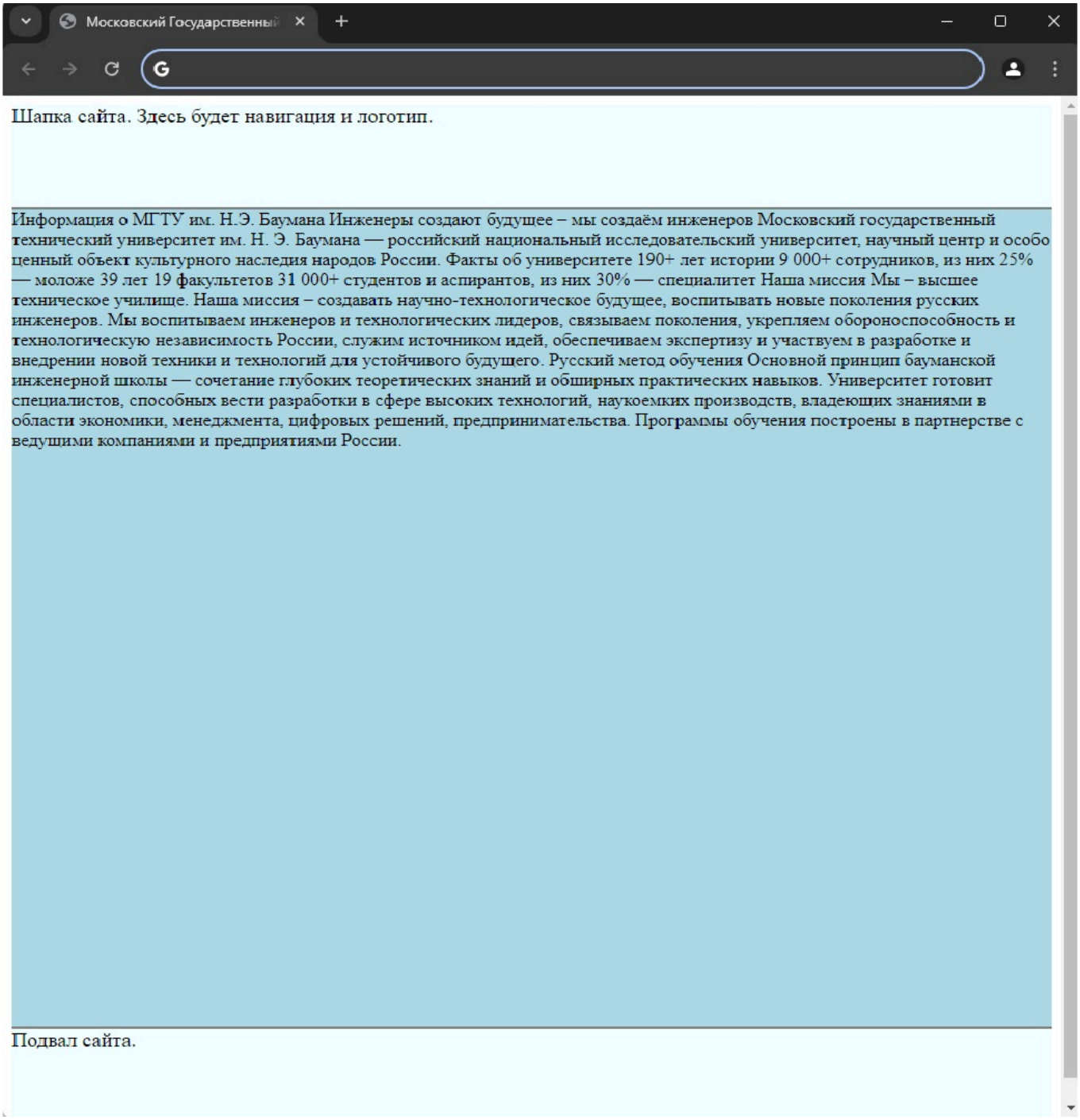


Рис. 14. Шапка, основная часть и подвал сайта о МГТУ.

### Разметка текста на веб-странице

Теперь пришло время заняться тем, что по-настоящему наполняет сайт — **разметкой текста**. Чтобы браузер понял, где на странице должен быть заголовок, где абзац, а где список — мы используем специальные HTML-теги.

Среди них особенно важны:

- **заголовки разделов** — `<h1>`–`<h6>`,
- **параграфы** — `<p>`,
- **списки** — `<ul>`, `<ol>`, `<li>`.

Разберёмся с каждым по порядку, а потом осуществим разметку текста вашего сайта с их помощью.

### Тэги заголовков разделов

Заголовки задают структуру текста и помогают ориентироваться на странице — как читателю, так и поисковым системам.

В HTML существует **шесть уровней заголовков**, которые представляют парные тэги `<h1>` `<h2>` `<h3>` `<h4>` `<h5>` `<h6>` .
`<h1>` — это заголовок самого верхнего уровня являющийся самым главным, а `<h6>` — самый мелкий и наименее значимый заголовок.

Обычно на одной странице используют **один** `<h1>`, чтобы обозначить её основную тему, а далее — заголовки уровней `<h2>`, `<h3>` и т. д. для подразделов.

Поисковые системы придают особое значение заголовкам. Тэг `<h1>` считается самым важным и называется заголовком верхнего уровня. Тэгом `<h1>` необходимо обрамлять текст, который в целом описывает содержание страницы.

Запомнить данные тэги очень просто: буква «h» в названии тэга — это первая буква английского слова «heading», которая означает «заголовок».
Пример использования тэга заголовка:

```
<h1> Информация о МГТУ им. Н.Э. Баумана </h1>
```

### Абзацы

Чтобы текст выглядел читаемо, его нужно делить на абзацы.
Для этого в HTML есть тег `<p>`, который означает *paragraph* — «параграф».
Пример использования тэгов параграфа:

```
<p> Инженеры создают будущее - мы создаём инженеров </p>
<p> Московский государственный технический университет им. Н. Э. Баумана —
российский национальный исследовательский университет,
```

```
научный центр и особо ценный объект культурного наследия народов России.
</p>
```

Каждый абзац — это отдельный блок текста, который браузер отобразит с отступом.

**Тэги списков**

Иногда нужно перечислить несколько пунктов — факты, шаги, характеристики. Для этого HTML предлагает **два типа списков**: упорядоченные и неупорядоченные.

**Неупорядоченный список**

Неупорядоченный список по-английски называется «unordered list». Для разметки такого списка используется тэг `<ul>`. Такой тэг используется, когда в списке порядок элементов не важен.

Внутри тэга `<ul>` может находиться только тэги `<li>` (сокращение от «list item» - в переводе с английского, элемент списка). Элементов списка может быть неограниченное количество, но должно быть не менее одного.
Пример использования тэга неупорядоченного списка:

```
<h2>Факты об университете</h2>

<ul>
  <li>190+ лет истории</li>
  <li>9 000+ сотрудников, из них 25% — моложе 39 лет</li>
  <li>19 факультетов</li>
  <li>31 000+ студентов и аспирантов, из них 30% — специалитет</li>
</ul>
```

В результате использования данных тэгов браузер вернет (Рис. 15):

**Факты об университете**

- 190+ лет истории
- 9 000+ сотрудников, из них 25% — моложе 39 лет
- 19 факультетов
- 31 000+ студентов и аспирантов, из них 30% — специалитет

Рис. 15. Использвание тэга неупорядоченного списка.

**Упорядоченный список**

Списки могут быть и упорядоченными («ordered list»). Для разметки упорядоченного списка используется тэг `<ol>`. Такой тэг используется, когда в списке порядок элементов важен.
Пункты упорядоченного списка тоже размечаются с помощью тега `<li>`.
Продемонстрируем использвание описываемого тэга.

```
<h2>Факты об университете</h2>

<ol>
  <li>190+ лет истории</li>
  <li>9 000+ сотрудников, из них 25% — моложе 39 лет</li>
  <li>19 факультетов</li>
  <li>31 000+ студентов и аспирантов, из них 30% — специалитет</li>
</ol>
```

В результате использования данных тэгов браузер вернет (Рис. 16):

## Факты об университете

1. 190+ лет истории
2. 9 000+ сотрудников, из них 25% — моложе 39 лет
3. 19 факультетов
4. 31 000+ студентов и аспирантов, из них 30% — специалитет

Рис. 16. Пример использования упорядоченного списка.

Пользуясь полученными знаниями, разметим текст нашего сайта:

```
<!DOCTYPE html>
<html lang="ru">
  <head>
    <link href="style.css" rel="stylesheet">
    <title>Московский Государственный Технический Университет им. Н.Э.
Баумана</title>
  </head>
  <body>
    <header>
        Шапка сайта. Здесь будет навигация и логотип.
    </header>
  <main>

    <h1>Информация о МГТУ им. Н.Э. Баумана</h1>

    <p>Инженеры создают будущее - мы создаём инженеров</p>

    <p>Московский государственный технический университет им. Н. Э. Баумана
— российский национальный исследовательский университет, научный центр и
особо ценный объект культурного наследия народов России.
    </p>

    <h2>Факты об университете</h2>
```

```
    <ul>
      <li>190+ лет истории</li>
      <li>9 000+ сотрудников, из них 25% — моложе 39 лет</li>
      <li>19 факультетов</li>
      <li>31 000+ студентов и аспирантов, из них 30% — специалитет</li>
    </ul>

    <h3>Наша миссия</h3>

    <p>
      Мы - высшее техническое училище. Наша миссия - создавать научно
технологическое будущее, воспитывать новые поколения русских инженеров.
      Мы воспитываем инженеров и технологических лидеров, связываем
поколения,
      укрепляем обороноспособность и технологическую независимость России,
      служим источником идей, обеспечиваем экспертизу и участвуем в
разработке
      и внедрении новой техники и технологий для устойчивого будущего.
    </p>

    <h4>
      Русский метод обучения
    </h4>

    <p>
      Основной принцип бауманской инженерной школы — сочетание глубоких
теоретических знаний и обширных практических навыков.
    </p>

    <p>
      Университет готовит специалистов, способных вести разработки в сфере
высоких технологий, наукоемких производств, владеющих знаниями в области
экономики, менеджмента, цифровых решений, предпринимательства.
    </p>

    <p>
      Программы обучения построены в партнерстве с ведущими компаниями и
предприятиями России.
    </p>

    <p>
      Основной принцип бауманской инженерной школы — сочетание глубоких
теоретических знаний и обширных практических навыков.
    </p>

    <p>
```

```
      Университет готовит специалистов, способных вести разработки в сфере
высоких технологий, наукоемких производств,владеющих знаниями в области
экономики, менеджмента, цифровых решений, предпринимательства.
    </p>

    <p>
      Программы обучения построены в партнерстве с ведущими компаниями и
предприятиями России.
    </p>

  </main>
  <footer>
    Подвал сайта.
  </footer>
  </body>
</html>
```

Обратим внимание, что для тэгов `h1-h6`, а также тэга `<p>` браузер принимает некий набор CSS-правил по умолчанию. Среди этих правил находятся те, что задают верхний (margin-top) и нижний отступ (margin-bottom) вокруг элемента. Такое поведение является нежелательным.

Для того, чтобы убрать верхний и нижний отступ, принудительно укажем в файле style.css, что для всех тэгов (селектор `*` выбирает все тэги) верхний и нижний отступ равен нулю. Если нам потребуется отступ, мы сами его определим для нужного элемента.

Таким образом, наш файл со стилями примет следующий вид:

```
* {
    margin: 0;
}

header {
    background-color: azure;
    font-size: 18px;
    border-bottom: 2px solid grey;
    height: 10vh;
}

main {
    background-color: lightblue;
    font-size: 16px;
    height: 80vh;
    padding:10px;
}
```

```css
h1, h2, h3, h4 {
    margin-top: 0;
    margin-bottom:0;
}

footer {
    background-color: azure;
    font-size: 18px;
    border-top: 2px solid grey;
    height: 10vh;
}
```

Давайте посмотрим как браузер отобразит нашу страницу (Рис. 17).

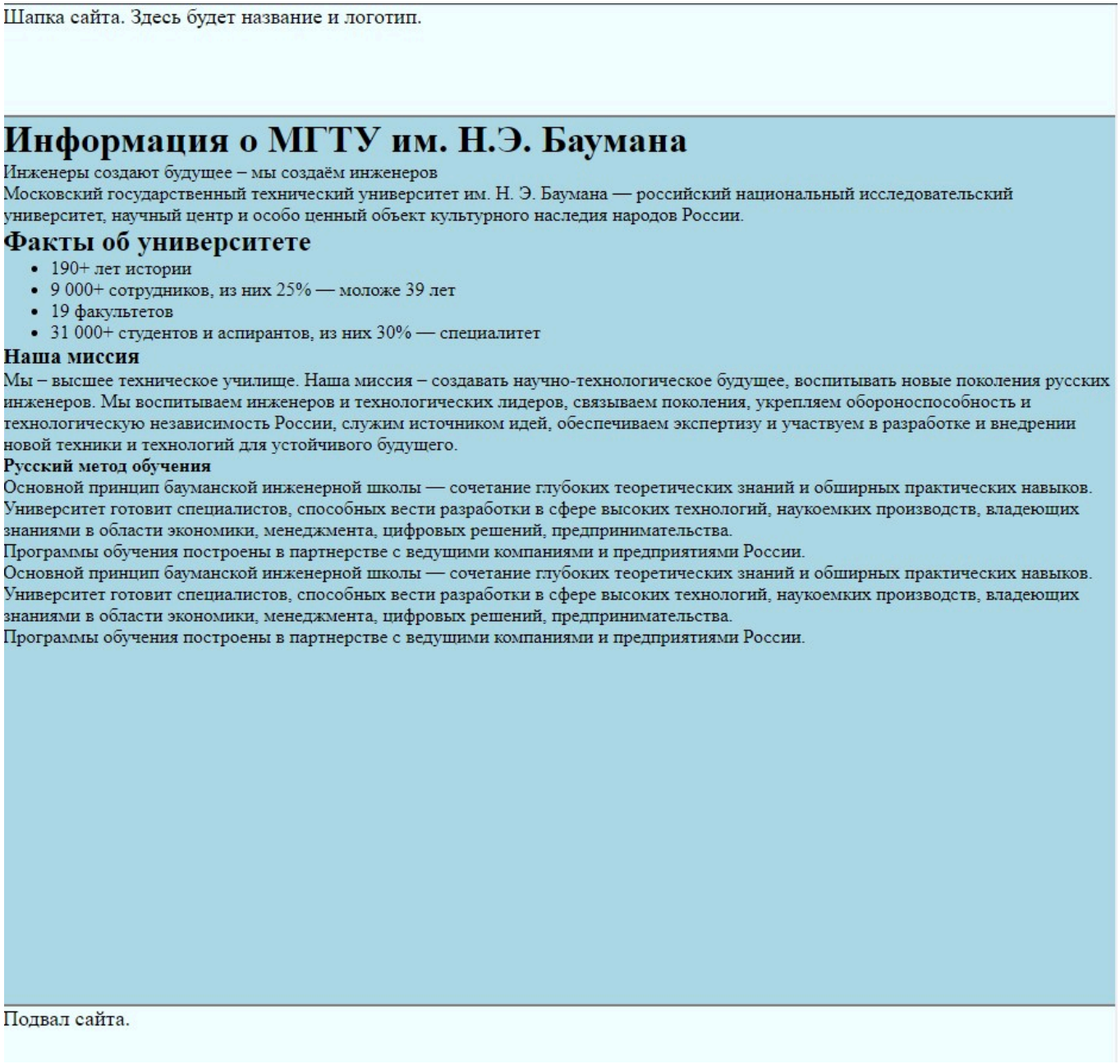



Рис. 17. Отображение нашей в браузере после верстки основного текста.

### Разметка шапки и основной части страницы

Пора добавить на наш сайт первые **реальные элементы интерфейса**: логотип и навигацию. Чтобы сделать это грамотно, познакомимся с несколькими важными тегами HTML и полезными приёмами CSS.

### Что такое `<div>` и зачем он нужен?

Тег `<div>` (сокращение от "division" — раздел) используется для **группировки элементов**.
Это как контейнер, в которую вы складываете связанные между собой части интерфейса — текст, изображения, списки или другие тэги.

С помощью данных тэгов разделим "шапку" на 2 раздела. У каждого тэга `<div>` есть свой атрибут `class`, описывающий назначение этого раздела и определяющий применяемые к нему CSS-правила:

```
<header>
    <div class="logo"></div>
    <div class="navigation"></div>
</header>
```

У нас получилось два контейнера внутри шапки сайта (`<header>`) :

- один для логотипа (`class="logo"`),
- второй для навигационного меню (`class="navigation"`).

### Атрибут `class` — ключ к стилям

Остановимся подробнее на атрибуте `class`.
В предыдущем блоке кода в тэге `<div>` используется атрибут `class` для добавления CSS-классов элементам HTML. Классы позволяют применять одни и те же стили CSS к разным элементам на странице.

Одному элементу можно присвоить один или несколько классов, разделяя их пробелами.

Рассмотрим пример. Мы описали три абзаца и добавили каждому тэгу `<p>` по атрибуту `class` . Обратите внимание, что в третьем тэге абзаца в атрибуте `class` указано 2 различных класса.

```
<p class="highlighted-text">
   Этот текст будет выделен
</p>

<p class="uppercase">
   Этот текст будет напечатан прописными буквами
</p>
```

```
<p class="highlighted-text uppercase">
    Этот текст будет напечатан прописными буквами и выделен
</p>
```

Допустим, что этому HTML соответствует следующий CSS:

```
.highlighted-text {
    background-color: lime;
    font-size: 18px;
  }

  .uppercase {
    text-transform: uppercase;
  }
```

Чуть ранее мы говорили о селекторах типа. Данные селекторы позволяют применять CSS-свойства к определенным HTML-тэгам.

Существуют также селекторы класса, позволяющие определять стиль элементов с определенным классом.

Например, это CSS-правило с помощью селектора класса выберет абзац с классом `.highlighted-text` и привяжет стилевые свойства `background-color: #4eff33;` (поменять цвет фона на лаймовый) и `font-size: 18px;` (поменять размер шрифта на значение в 18 px) к выбранному элементу:

```
.highlighted-text {
    background-color: lime;
    font-size: 18px;
  }
```

Отметим, что если элементу присвоено несколько классов через пробел, например как этому:

```
<p class="highlighted-text uppercase">
    Этот текст будет напечатан прописными буквами и выделен
</p>
```

То к нему по очереди применится два CSS-правила. В данном случае, текст будет выделен и напечатан прописными буквами.

Посмотрим, как браузер отобразит наш пример с тремя абзацами (Рис.18)

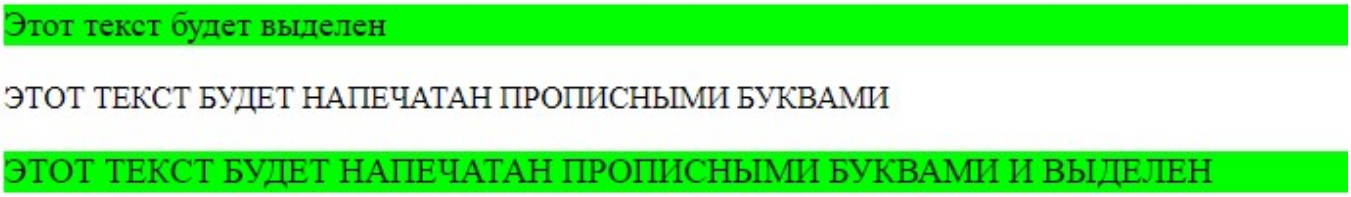



Рис.18. Применение селектора класса

**Разметка шапки и основной части. Продолжение.**

Давайте вставим логотип МГТУ в "шапку" нашего сайта. Для этого познакомимся с еще одним тэгом.

**Добавляем логотип:** `<img>`

Данный тэг используется для вставки изображения на веб-страницы.
Тэг `<img>` является одинарным. Это означает, что у него нет закрывающего тэга.
Описываемый тэг поддерживает изображения в различных форматах, таких как JPEG, PNG, WebP, GIF, SVG.

У данного тэга есть обязательные атрибуты. Перечислим их:

- `src` — указывает путь к изображению
- `alt` — обеспечивает текстовое описание изображения

Атрибуты `width` и `height` рекомендуются для определения размеров изображения.

Изображения вы можете найти в папке с материалами к нашему курсу.

Применим данный тэг в нашем проекте в двух местах:

В шапке:

```
<header>
   <div class="logo">
      <img class="logo__image" src="img/logo_with_text.png" alt="Логотип
МГТУ" width="150" height="150">
   </div>

    <div class="navigation"></div>

</header>
```

и в контентной части главной страницы нашего сайта:

```
...
<div class="about">
      <div class="about__text">
        <h2>Факты об университете</h2>
        <ul>
          <li>190+ лет истории</li>
          <li>9 000+ сотрудников, из них 25% — моложе 39 лет</li>
          <li>19 факультетов</li>
          <li>31 000+ студентов и аспирантов, из них 30% — специалитет</li>
        </ul>
      </div>
      <div class="about__image-wrapper">
        <img class="about__image" src="img/Logo_Black_No-BG.png"
alt="Логотип МГТУ" width="100" height="100">
      </div>
    </div>
...
```

Пояснения к коду выше. В контентной части мы поместили все, что относится к фактам об университете в `<div class="about">`.

Так мы заложили фудамент для группировки смысловой части страницы в единый раздел.

Внутри мы хотим сделать следующее - слева должны быть факты об университете, а справа большой логотип МГТУ. Для достижения этой цели мы поместили внутрь раздела `<div class="about">` следующее:

- `<div class="about__text">` - внутри этого отдела мы разместили наш старый код с неупорядоченным списком.
- `<div class="about__image-wrapper">` - внутри этого раздела мы поместили изображение с логотипом с помощью тэга `<img>`

Посмотрим как преобразилась наша веб-страница (Рис. 19):

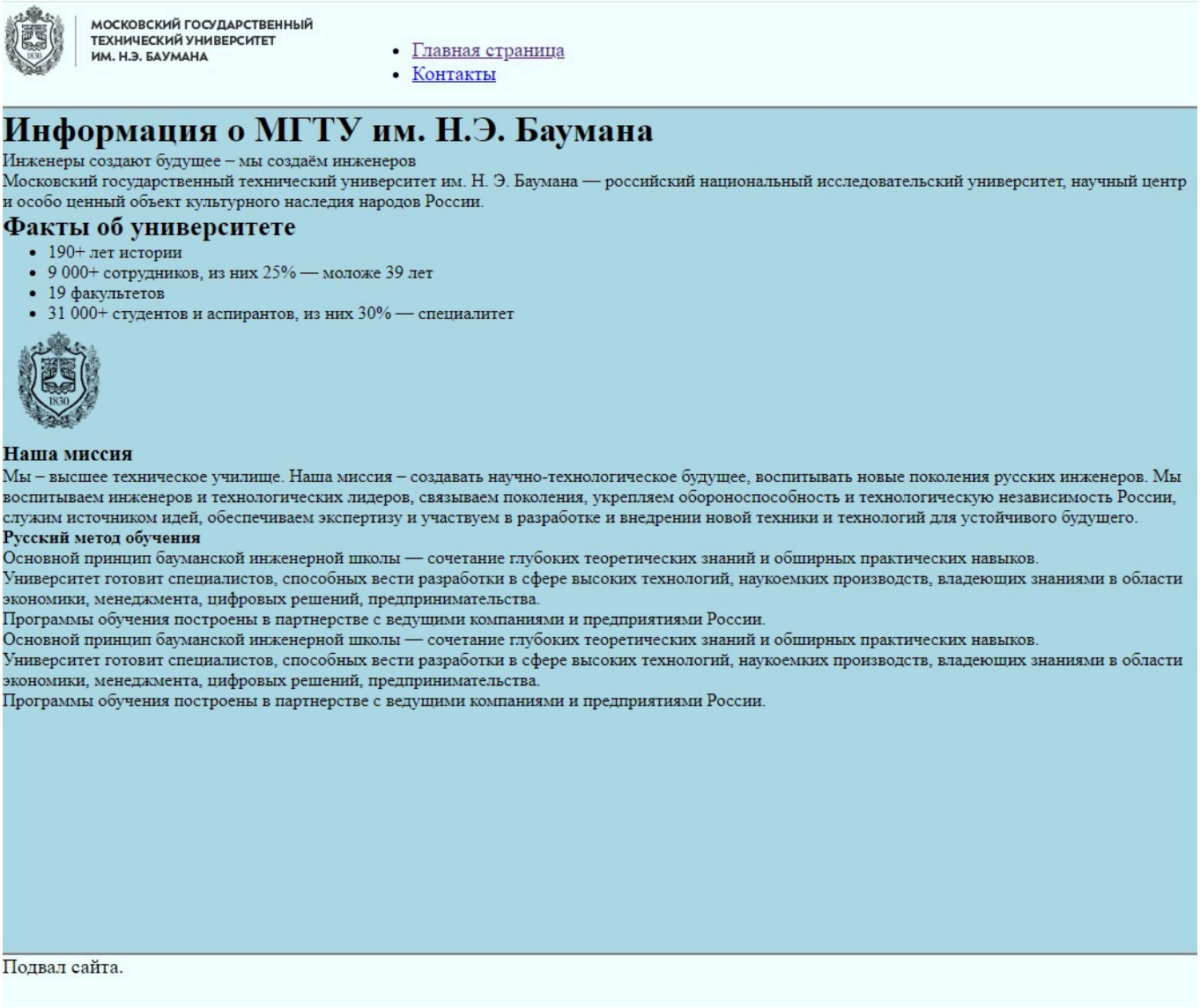



Рис. 19. Страница с добавленными изображениями.

### Гиперссылки в HTML: тег `<a>`

Одним из ключевых механизмов, лежащих в основе Всемирной паутины, является возможность связывать между собой различные веб-страницы и ресурсы. Для этого в языке HTML используется **тег гиперссылки** — `<a>`.

Тег `<a>` (от англ. *anchor* — якорь) предназначен для создания ссылок. HTML-ссылки позволяют пользователю:

- переходить между страницами одного сайта;
- открывать внешние сайты;
- скачивать файлы;
- перемещаться по частям одной и той же страницы.

Простейшая ссылка оформляется следующим образом:

```
<a href="https://bmstu.ru">Официальный сайт МГТУ</a>
```

В данном примере `href` — это атрибут, задающий **URL-адрес** ресурса, на который будет происходить переход.

Иногда `<a>` используется без атрибута `href`. Такая конструкция называется **ссылкой-заглушкой** и используется как временный элемент навигации:

```
<a href="#">Контакты</a>
```

Символ `#` указывает на начало текущей страницы, и перехода фактически не происходит. В дальнейшем атрибут `href` может быть заменён на реальный адрес.

Вооружившись знанием про ссылки, заполним блок с навигацией. На нашем сайте в навигации будет 2 страницы:

- главная страница (ссылка на файл `index.html`);
- страница с контактами МГТУ (разместим ссылку-заглушку);

В HTML-навигации часто используют неупорядоченные списки (`<ul>`) в сочетании с элементами списка (`<li>`) и ссылками (`<a>`), что обеспечивает логичную, расширяемую структуру и совместимость с системами чтения с экрана (screen readers) и поисковыми системами.

```
<div class="navigation">
   <ul>
     <li>
        <a href="index.html">Главная страница</a>
      </li>
      <li>
         <a href="#">Контакты</a>
      </li>
   </ul>
</div>
```

После добавления данного HTML-кода мы увидим в шапке сайта две активные текстовые ссылки. Однако, как показано на рисунке 20, возникает проблема — навигационное меню **"наползает" на основное содержимое страницы**.

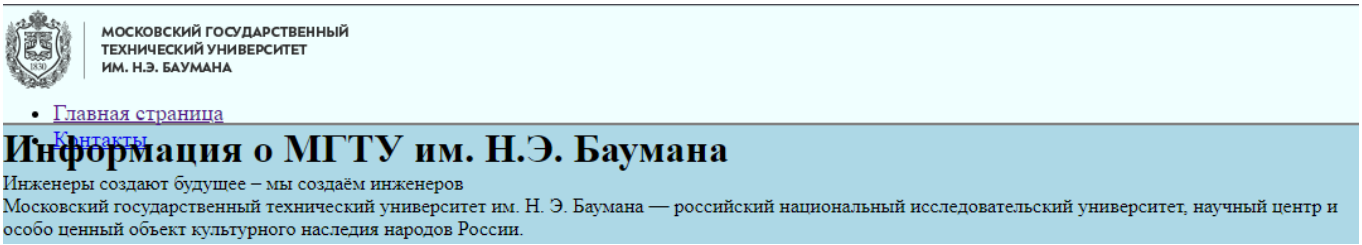



Рис. 20. "Шапка" с навигацией. Навигация "наползла" на контентную часть.

То, что вы видирте в браузере иллюстрирует проблему наложения элементов. Это поведение связано с тем, **как браузер по умолчанию интерпретирует и размещает HTML-элементы на странице**

Для решения подобных визуальных конфликтов важно понимать:

- какие существуют **типы HTML-элементов**,
- как работает **модель потока документа**,
- как CSS позволяет управлять размещением и выравниванием элементов.

Этими вопросами мы и займёмся в следующем разделе.

### Отображение HTML-элементов в браузере: поток документа и типы визуального представления

Одним из фундаментальных понятий в веб-разработке является модель потока документа, определяющая порядок отображения HTML-элементов в браузере. Понимание данной модели критически важно для управления структурой и оформлением веб-страниц с использованием CSS.

**Поток документа** описывает, как браузер размещает элементы на странице в зависимости от их типа и CSS-свойств. Различают три основных типа потока:

1. **Поток блочных элементов (Block Formatting Context)** — элементы размещаются вертикально, начиная с новой строки, и по умолчанию занимают всю доступную ширину родительского контейнера. К таким элементам относятся, например, `<div>`, `<p>`, `<section>`, `<h1>`–`<h6>` и другие.
2. **Поток встроенных элементов (Inline Formatting Context)** — элементы размещаются внутри строк текста и не создают новой строки. Типичными представителями являются `<a>`, `<span>`, `<em>`, `<strong>` и др.
3. **Поток замещаемых элементов (Replaced Elements)** — элементы, содержимое которых не определяется HTML-разметкой напрямую, а подставляется браузером. Примеры: изображения (`<img>`), видео (`<video>`), `<iframe>` и т.д. Эти элементы могут вести себя как блочные или строчные в зависимости от контекста.

Каждый тип потока имеет свои правила и характеристики, которые определяют, как элементы будут отображаться на странице.

Каждому HTML-элементу по умолчанию назначается определённый **тип визуального представления**. Однако этот тип может быть переопределён с помощью CSS-свойства `display`.

В CSS существует 12 основных типов отображения элементов. Перечислим их: `block`, `inline`, `inline-block`, `flex`, `grid`, `inline-flex`, `inline-grid`, `table`, `table-cell`, `table-row`, `list-item`, `none`.

Среди множества возможных значений `display` рассмотрим три базовых, необходимых для понимания структуры страницы:

Рассмотрим три типа визуального представления элементов— **блочный (`block`)**, **строчный (`inline`)** и **строчно-блочный (`inline-block`)**. Эти характеристики относятся к поведению блоков в контексте потока документа и относительно других блоков на странице.

Если элемент определён как **блочный**, то он будет вести себя следующим образом:

- Начинается с новой строки. Иными словами, несколько блочных элементов располагаются один над другим, вертикально.
- Будет расширяться вдоль строки таким образом, чтобы заполнить всё пространство, доступное в её контейнере (объемлющем блоке). В большинстве случаев это означает, что блок станет такой же ширины, как и его контейнер, заполняя 100% доступного пространства.
- Будут применяться атрибуты `width` и `height`
- Внешние и внутренние отступы, рамка будут отодвигать от него другие элементы.

Если не изменить намеренно тип отображения на строчный, то такие элементы, как заголовки (например, `<h1>`) и `<p>`, все используют `block` как свой внешний тип отображения по умолчанию.

Если элемент имеет тип отображения **строчный**, то:

- Он не будет начинаться с новой строки. Иными словами, элементы располагаются на той же строке, последовательно.
- Не будут применяться атрибуты `width` и `height`. Ширина и высота элемента будут определяться по содержимому.
- Вертикальные внешние и внутренние отступы, рамки будут применяться, но не будут отодвигать другие строчные элементы.
- Горизонтальные внешние и внутренние отступы, рамки будут применяться и будут отодвигать другие строчные элементы.

Элемент `<a>`, используемый для ссылок — это пример строчного элемента.

Тип отображения **inline-block** является комбинацией двух других типов отображения - **inline** и **block**. Этот тип отображения позволяет элементу вести себя как блочный элемент, но при этом занимать только ту ширину, которая необходима для отображения содержимого.

Вернёмся к задаче, возникшей при вёрстке "шапки" сайта. В текущей разметке два элемента — логотип и навигация — оформлены как блочные (`<div>`). По умолчанию они располагаются **друг под другом**, то есть вертикально:

```
<header>
   <div class="logo">
      <img class="logo__image" src="img/logo_with_text.png" alt="Логотип
МГТУ" width="275" height="70">
   </div>

    <div class="navigation"></div>

</header>
```

Чтобы визуально выстроить данные элементы **в одну строку**, необходимо изменить их поведение в потоке с помощью свойства `display: inline-block` и задать соответствующую ширину.

```
.logo {
    display: inline-block;
    width: 30%;
}

.navigation {
    display: inline-block;
    width: 65%;
}
```

Обратим внимание, что в данном примере ширина (`width`) задается в процентах. Размер в процентах берется относительно ширины родительского элемента, иными словами HTML-элемента, в котором находится тэг, к которому применяется CSS-свойство.

Это позволяет адаптировать элементы под размеры экрана различных устройств.

Посмотрим как изменилась "шапка" нашего сайта. Элементы выстроились в строку. (Рис. 21):

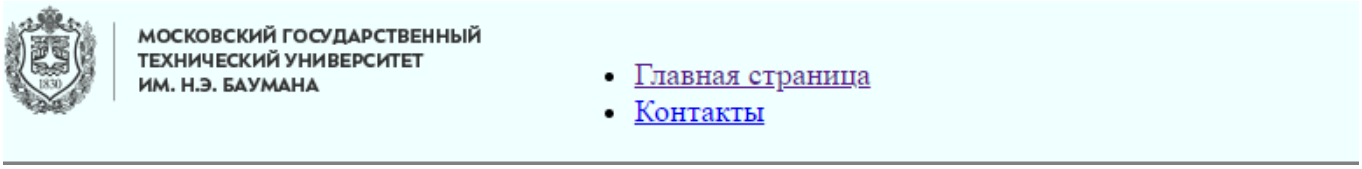



Рис. 21. Шапка нашего сайта после изменений.

Проделаем то же самое для вложенных `<div>` секции с фактами об университете (`<div class="about">`).

```
.about__text {
    display: inline-block;
    width: 45%;
}

.about__image-wrapper {
    display: inline-block;
    width: 45%;
}
```

Посмотрим как изменился сайт после внесенных изменений (Рис. 22)

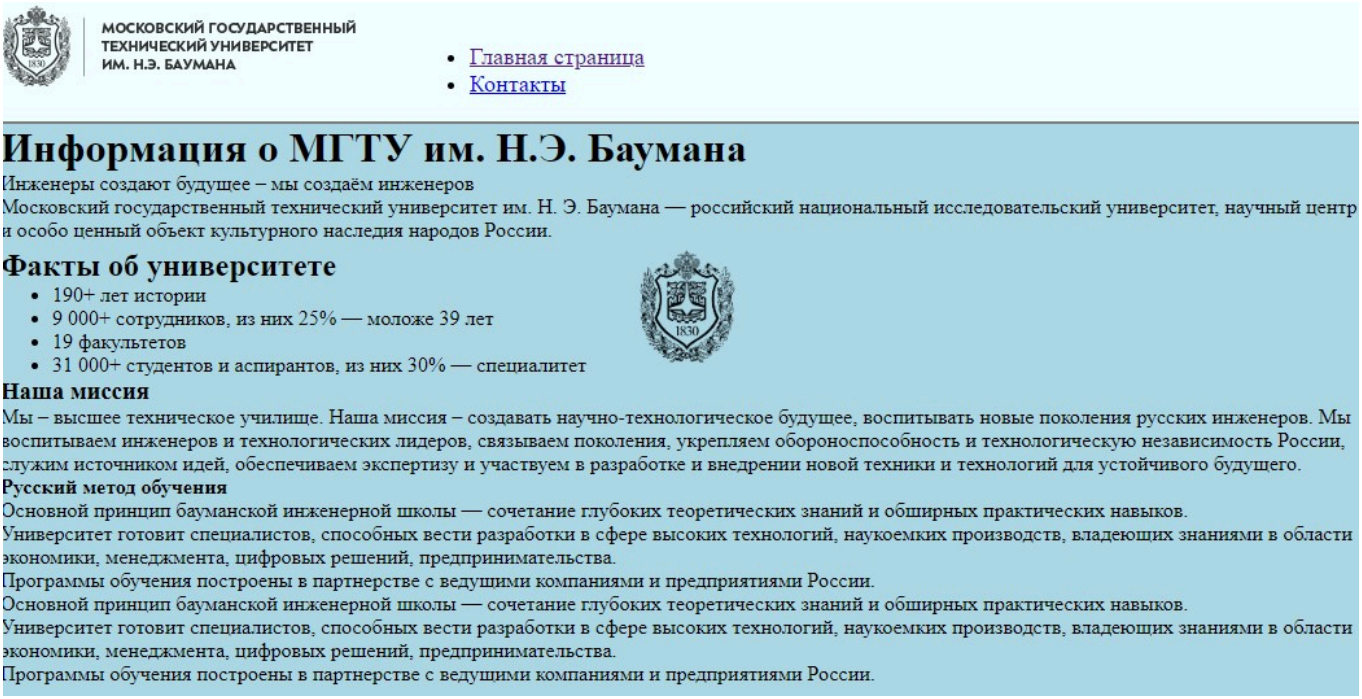


Рис. 22. Внешний вид главной страницы сайта после внесения изменений.

### Завершение вёрстки навигационного блока

Сейчас наша задача завершить верстку навигационного блока.
На данный момент он представляет собой неупорядоченный список.
Добавим новый код в проект и прокомментируем назначение каждой новой сущности.

Для повышения читаемости и поддержки кода, особенно в масштабных проектах, каждому элементу рекомендуется присваивать **семантически осмысленные CSS-классы**. Используем модифицированную нотацию **BEM** (Block–Element–Modifier), которая обеспечивает однозначность и иерархичность именования.

В результате структура навигационного блока может выглядеть следующим образом:

```
<div class="navigation">
   <ul class="navigation__list">
      <li class="navigation__list-item">
         <a class="navigation__link" href="index.html">Главная страница</a>
       </li>
       <li class="navigation__list-item">
         <a class="navigation__link" href="#">Контакты</a>
       </li>
   </ul>
</div>
```

Пояснения к именованию классов в данном примере:

- тэг `<ul>` - это тэг неупорядоченного списка( `list` ), поэтому мы даем ему класс `navigation__list` (написание через двойное нижнее подчеркивание)
- тэг `<li>` - это тэг элемента списка `list item` , поэтому мы даем ему класс `navigation__list-item`
- тэг `<a>` - это тэг элемента списка `link` , поэтому мы даем ему класс `navigation__link`

Давать элементам классы и называть их подобным понятным образом - крайне хорошая практика, существенно улучшающая читаемость файлов со стилями, которые в реальных проектах могут содержать десятки тысяч строк.

Далее, с помощью CSS-правил отключим маркеры для элементов списка, выстроим из в строку и красиво расположим их на странице:

```
header {
    background-color: azure;
    font-size: 18px;
    border-bottom: 2px solid grey;
    height: 10vh;
}

.logo {
    display: inline-block;
    width: 30%;
    vertical-align: middle;
}

.navigation {
    display: inline-block;
    width: 60%;
    vertical-align: middle;
}

.navigation__list {
    width: 100%;
    list-style-type: none;
    text-align: right;
}

.navigation__list-item {
    display: inline-block;
    width: 20%;

}
```

Прокомментируем подробнее данный код:

- элементам списка с классами `navigation__list-item` задали тип отображения `inline-block`, чтобы выстроить их в строку;
- с помощью CSS-свойства `list-style-type: none` примененное к списку с классом `navigation__list` мы избавились от маркеров списка (точек перед элементами списка);
- с помощью CSS-свойства `text-align: right` мы сделали горизонтальное выравнивание в правую строну содержимого элемента с классом `navigation__list`;
- мы задали вертикальное выравнивание с помощью CSS-свойства `vertical-align: middle` внутри тэга `header` для элементов с классами `logo` и `navigation__list`.

Изменим внутренние отступы нашего логотипа настроим с помощью внутренних отступов (`padding`), что сделает визуальное представление более аккуратным и уравновешенным.

```
.logo__image {
    padding-top: 10px;
    padding-left: 20px;
}
```

Посмотрим как теперь выглядит "шапка" нашего сайта (Рис. 23)

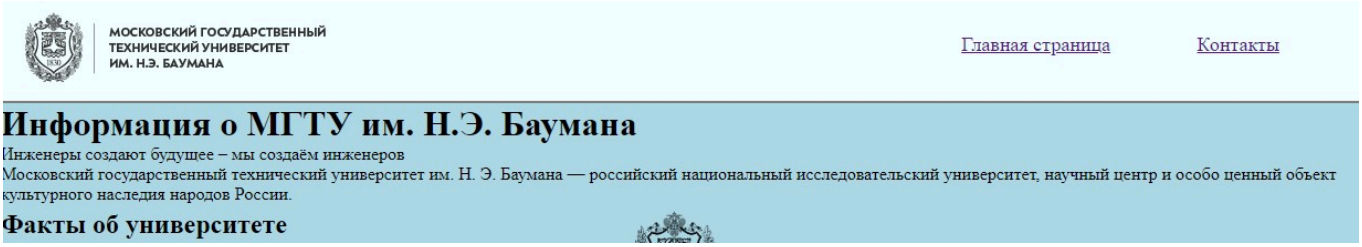


Рис. 23. Обновленная шапка сайта

Поздравляем. Первая страница завершена. Двигаемся ко второй.

## Формы и обработка пользовательского ввода

В современных веб-приложениях интерактивность является одним из ключевых аспектов. Статичная информация уступает место диалоговому взаимодействию между пользователем и системой. На этом этапе мы познакомимся с основами HTML-форм, элементами пользовательского ввода, а также с их базовой стилизацией при помощи CSS.

Настало время создать вторую страницу сайта — страницу обратной связи, содержащей форму, через которую пользователь сможет отправить сообщение администратору.

Современные сайты не просто показывают статичную информацию, они стараются взаимодействовать с пользователями.
Но как же организуется взаимодействие с пользователем на веб-странице?

Получать пользовательские данные помогают специальные HTML элементы под названием формы.

**Форма** в HTML — это структурный элемент веб-страницы, предназначенный для ввода, обработки и отправки данных. Формы используются для получения различной информации от пользователя, например регистрационных данных, сообщениий, комментариев, параметров поиска и многого другого.

Форма создается с помощью тега `<form>` и может содержать различные элементы ввода, такие как текстовые поля, флажки, переключатели, выпадающие списки и кнопки.

Приведем пример простой формы в HTML:

```
    <div class="contacts__wrapper">
        <div class="contacts__header">
            <h1>Связаться с нами</h1>
        </div>
        <form class="contacts__form" action="/submit_form.php"
method="post">
            <label class="contacts__form-label" for="username">Как вас
зовут?</label>
            <input class="contacts__form-input" type="text" id="username"
name="username"><br><br>
            <label class="contacts__form-label" for="message">Сообщение
</label>
            <textarea class="contacts__form-textarea"  rows="5"
name="message" placeholder="Расскажите обо всём, что для вас важно"
id="message"></textarea>
            <br><br>
            <input class="contacts__form-submit" type="submit"
value="Отправить">
          </form>
    </div>
```

Создайте в папке с вашим сайтом (в нашем случае эта папка называется BMSTU_WEBSITE) файл contacts.html

Наполним созданный файл. Для начала скопируйте в файл "Шапку" и "Подвал" сайта. Между шапкой и подвалом мы разместим наш основной контент внутри тэга `<main>`.

Скопируйте данную форму внутрь тэга `<main>` страницы contacts.html
Откройте браузер чтобы посмотреть что получилось (Рис. 24)

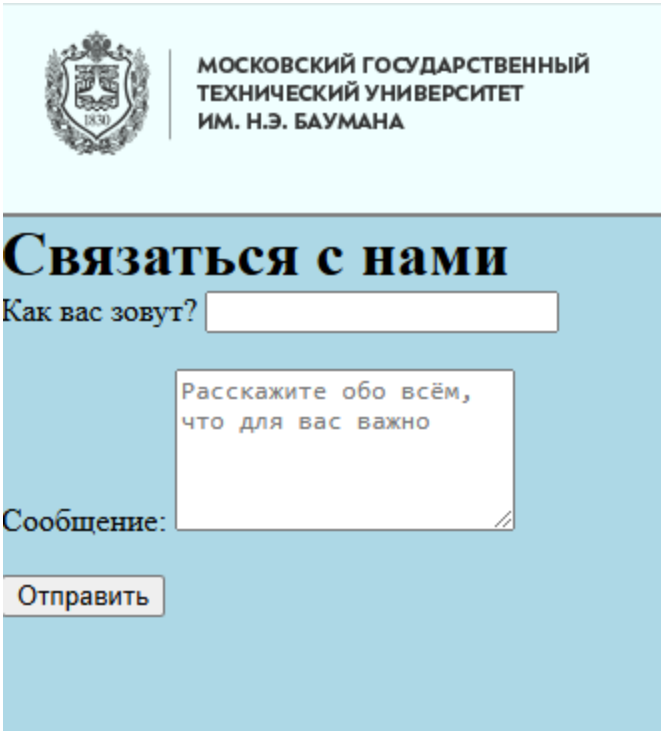



Рис.24. Форма для пользовательского ввода появилась на сайте.

Пока выглядит не очень красиво.

Добавим в файл `style.css` стили, обеспечивающие корректное отображение и базовую визуальную эстетику формы:

```
.contacts__wrapper {
    width: 30%;
    margin: 0 auto;
    padding-top: 30px;
    text-align: center;
}

.contacts__header {
    padding-bottom: 30px;
}

.contacts__form-label {
    font-size: 16px;
    font-weight: bold;
}

.contacts__form-input,.contacts__form-textarea {
    margin-top: 10px;
}

.contacts__form-label,
.contacts__form-input,
.contacts__form-textarea {
    width: 100%;
```

```
}

.contacts__form-submit {
    background-color: blue;
    color: white;
    padding: 10px 12px;
    border-radius: 10px;
    border: 1px solid blue;
    font-size: 16px;
    font-weight: bold;
}
```

Откройте страницу в браузере, чтобы убедиться, что форма отобразилась корректно. (Рис.25)

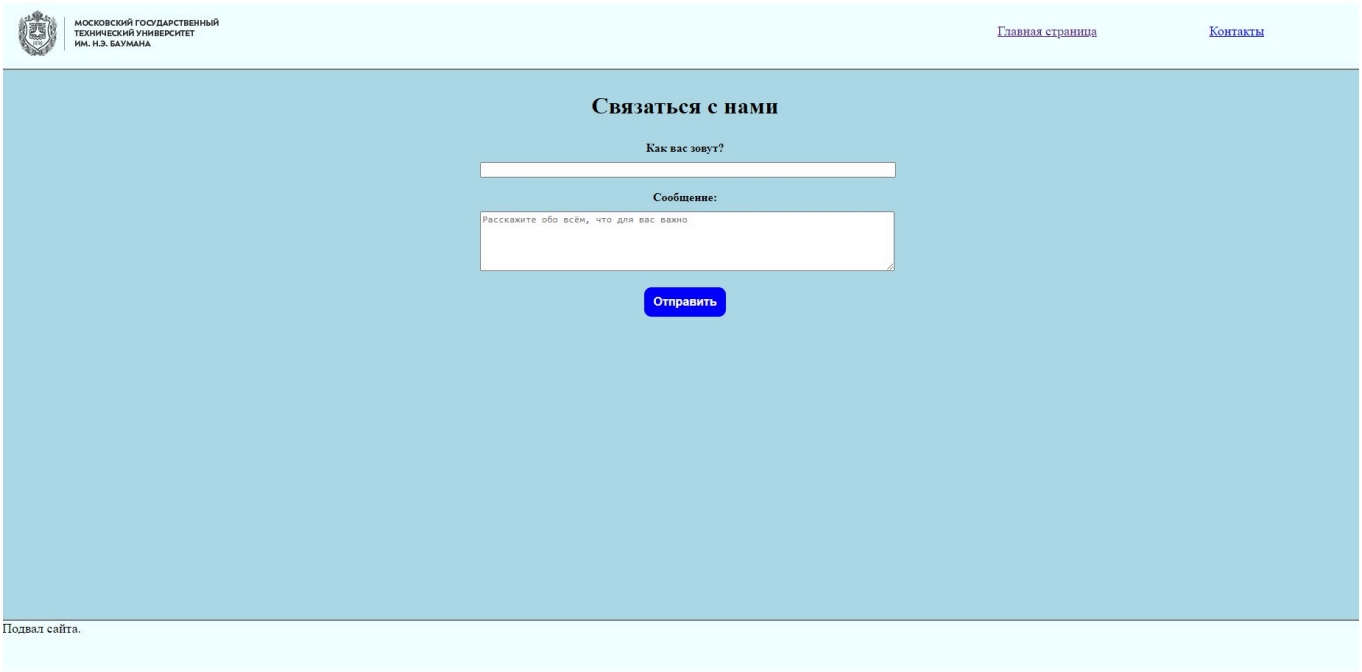



Рис.25. Стилизованая форма пользовательского ввода.

Разберем структуру формы
Каждая форма состоит из нескольких основных элементов:

1. Тега `<form>`. Обрамляет всю структуру формы и определяет, каким способом и куда отправляются данные.

   Атрибуты:
   - `action` — путь к серверному скрипту, который будет обрабатывать отправленные данные;
   - `method` — метод передачи данных (`GET` или `POST`).
2. Элементов ввода (`input`, `textarea`, `select` и т. д.)
   Эти элементы позволяют пользователям вводить данные в форму. Например, поле для ввода текста (`<input type="text">`), поле для ввода пароля (`<input`

`type="password">`), текстовое поле (`<textarea>`) и выпадающий список (`<select>`).
3. Элемента `<label>`. Этот элемент используется для связывания метки с элементом ввода через атрибут `for`, значение которого должно совпадать с `id` соответствующего поля. Метка представляет собой текстовую подпись для элемента ввода, что делает форму более понятной для пользователя.
4. Элемента `<input type="submit">`: Этот элемент создает кнопку в форме, которую пользователь может нажать для отправки данных или выполнения какого-либо действия.
5. Других атрибутов: В HTML форма может содержать и другие атрибуты, такие как name, id, placeholder, required и другие, которые позволяют настраивать поведение и внешний вид элементов формы.

HTML форма является основным способом взаимодействия пользователей с веб-страницами и передачи данных на сервер для обработки. При помощи CSS и JavaScript можно дополнительно стилизовать и улучшить функциональность формы.

### Важное замечание о безопасности

**Пользовательский ввод — это потенциально небезопасный источник данных.**

Веб-приложение не должно "доверять" вводимым значениям: они могут быть произвольными, намеренно искажёнными, содержать вредоносный код или вызывать непредсказуемое поведение.

Перед обработкой такие данные обязательно должны проходить:

- валидацию (проверку корректности);
- экранирование специальных символов;
- защиту от атак типа XSS и SQL-инъекций.

Мы подробно разберём аспекты обработки форм и обеспечения безопасности при изучении серверной стороны (в частности, языка **PHP**) и **работы с базами данных**.

## Javascript

### Роль JavaScript в веб-разработке

Познакомимся с еще одной популярнейшей веб-технологией под названием javascript.

На современном этапе развития веб-технологий **JavaScript** занимает ключевое положение в тройке основных инструментов фронтенд-разработки наряду с HTML и CSS. Если HTML определяет **структуру**, а CSS — **внешнее оформление** веб-страницы, то JavaScript отвечает за её **динамическое поведение** и **взаимодействие с пользователем**.

JavaScript позволяет создавать интерактивные веб-приложения, которые реагируют на действия пользователя без необходимости перезагружать страницу. Именно благодаря JavaScript на сайтах реализуются такие элементы, как выпадающие меню, модальные окна, слайдеры, калькуляторы, валидация форм и многое другое.

## Как работает javascript

JavaScript является **интерпретируемым языком программирования**, что означает, что его код выполняется **пошагово** непосредственно во время работы программы, а не компилируется заранее в машинный код.

Интерпретация происходит на стороне клиента, то есть **в браузере пользователя**. Таким образом, написанный на JavaScript код поступает от сервера в исходном виде и исполняется браузером на устройстве пользователя.

## Интерпретаторы JavaScript в различных браузерах

Каждый современный браузер содержит встроенный интерпретатор JavaScript:

- **Google Chrome** — движок **V8** (один из самых производительных);
- **Mozilla Firefox** — **SpiderMonkey**;
- **Apple Safari** — **JavaScriptCore** (известен также как Nitro);
- **Microsoft Edge** (на базе Chromium) — также использует **V8**;
- (старые версии Edge — движок **Chakra**).

Эти движки постоянно совершенствуются и оптимизируются для повышения скорости исполнения и безопасности JavaScript-кода.

## Взаимодействие HTML, CSS и JavaScript

Понимание взаимодействия между тремя основными веб-технологиями является основополагающим для любого веб-разработчика:

- **HTML (HyperText Markup Language)** — отвечает за **структуру** веб-страницы: размещение заголовков, абзацев, изображений, списков и других элементов.
- **CSS (Cascading Style Sheets)** — управляет **визуальным оформлением** страницы: цветами, шрифтами, размерами, отступами, выравниванием и анимациями. CSS использует селекторы, чтобы применять стили к определённым HTML-элементам.
- **JavaScript** — обеспечивает **логическое поведение** страницы: обработку событий (например, нажатие кнопок), динамическое обновление содержимого, валидацию пользовательского ввода, взаимодействие с сервером без перезагрузки страницы (через технологии AJAX и Fetch API) и многое другое.

> Таким образом, можно представить взаимодействие трёх технологий как три уровня:
>
> - **HTML** — каркас и содержание,
> - **CSS** — внешний вид и оформление,
> - **JavaScript** — интерактивность и логика поведения.

### Подключение JavaScript к HTML-документу

Прежде чем приступить к практическому использованию языка JavaScript, необходимо разобраться, как именно скрипты интегрируются в HTML-страницу. Существует два

основных способа подключения JavaScript-кода:

- Встраивание кода JavaScript непосредственно в HTML-документ:
  Код JavaScript может быть размещён прямо внутри HTML-файла в специальном теге `<script>`. Такой подход удобен для простых скриптов или демонстрационных примеров:

```
<script>
  alert("Привет, мир!");
</script>
```

Такой код можно размещать как в разделе `<head>`, так и перед закрывающим тегом `</body>`. Однако предпочтительнее — в конце тела документа, чтобы не блокировать отображение страницы.

- Подключение внешнего JavaScript-файла
  Более распространённым и рекомендуемым способом является подключение внешнего файла с JavaScript-кодом. Для этого также используется тег `<script>`, но в него добавляется атрибут `src`, указывающий путь к соответствующему `.js`-файлу:

```
<script src="script.js"></script>
```

Подключение внешнего скрипта повышает читаемость и модульность проекта, позволяет использовать один и тот же код на нескольких страницах и облегчает сопровождение веб-приложения. Именно этот подход мы будем применять в рамках нашего курса.

**Первая программа на JavaScript**

Для начала практики создадим файл с JavaScript-кодом и подключим его к нашей веб-странице.

**Шаг 1. Создание внешнего скрипта**

Создайте в директории с нашим сайтом новый файл с именем `script.js` и поместите в него следующий код:

```
let message = "Привет, мир!";
alert(message);
```

этой программе:

- мы создаём **переменную** `message` и присваиваем ей текстовое значение `"Привет, мир!"`;
- затем вызываем функцию `alert(message)`, которая выводит всплывающее окно с содержимым переменной `message`.

> Обратите внимание: переменная объявляется с помощью ключевого слова `let`, что означает создание новой области хранения в памяти, доступной в текущем блоке кода.

### Шаг 2. Подключение скрипта к HTML-странице

Откройте файл `contacts.html` и вставьте следующий HTML-код перед закрывающим тегом `</body>`:

```
  <footer>
    Подвал сайта.
  </footer>

 <script src="script.js"></script>
  </body>
</html>
```

Размещение `<script>` внизу тела документа позволяет браузеру сначала отобразить весь HTML-контент, а затем выполнить скрипт — это хорошая практика для повышения производительности.

### Шаг 3. Запуск скрипта

Сохраните файл и откройте страницу contacts.html в браузере (Рис.26).

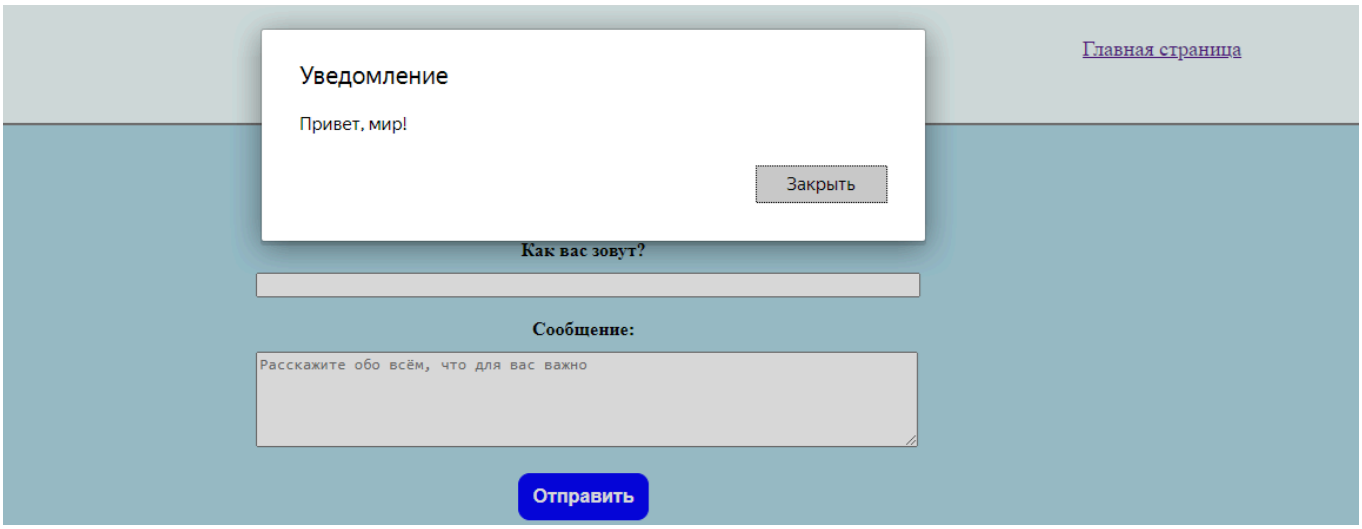



Рис.26. Результат выполнения первой программы на javascript.

Вы видите уведомление на всплывающем окне, на котором написано "Привет, мир!" .
Поздравляем! Ваша первая программа на javascript готова.
Это всплывающее окно - результат работы функции `alert()`.
Содержание уведомления мы передали в функцию с помощью переменной message, которую мы создали с помощью ключевого слова `let` и оператора присваивания `=`

## Основы синтаксиса javascript

В этом разделе мы рассмотрим базовые синтаксические конструкции языка JavaScript — одного из ключевых инструментов веб-разработки. Освоив их, вы сможете уверенно читать, понимать и писать простой JavaScript-код, необходимый для создания интерактивного поведения веб-страниц.

### Инструкции

Код на JavaScript состоит из **инструкций** — отдельных команд, которые выполняются браузером последовательно. Каждая инструкция завершается точкой с запятой (`;`), хотя в простых случаях она может быть опущена:

```
alert("Привет, мир!");
```

Эта инструкция вызывает встроенную функцию `alert()`, отображающую всплывающее окно с переданным сообщением.

### Комментарии

**Комментарии** служат для пояснения кода и игнорируются интерпретатором JavaScript. Они бывают двух видов:

- **Однострочные комментарии** начинаются с `//`:

```
// Это однострочный комментарий
let x = 5; // комментарий после кода
```

- **Многострочные комментарии** заключаются между `/*` и `*/`:

```
/*   Это многострочный

комментарий */

let y = 10;
```

### Переменные

Переменные в JavaScript — это контейнеры для хранения данных. Вы можете думать о них как о ящиках, на которых есть имена. Переменные объявляются с помощью ключевых слов `var`, `let` и `const`.

В рамках данного курса мы будем использовать только `let`, как современный и предпочтительный способ объявления переменных.

### Типы данных

Значение в JavaScript всегда относится к данным определённого типа. Например, это может быть строка или число.

Тип данных в языке программирования определяет допустимые значения, а также допустимые операции с данными этого типа.

Например, с данными типа «целое число» допускается выполнять арифметические операции, а с данными типа «строка» — нет. Поделить строку на строку невозможно.

Есть восемь основных типов данных в JavaScript. В данном курсе мы рассмотрим два: числа и строки.

Важно понимать, что переменная в JavaScript может содержать любые данные.
В один момент там может быть строка, а в другой – число:

```
let message = "МГТУ им. Н.Э. Баумана";
message = 1830;
```

Языки программирования, в которых такое возможно, называются «динамически типизированными». Это значит, что типы данных есть, но переменные не привязаны ни к одному из них.

**Тип данных число**

Числовой тип данных (`number`) представляет как целочисленные значения, так и числа с плавающей точкой.

Существует множество операций для чисел, например, умножение `*`, деление `/`, сложение `+`, вычитание `-` и так далее.

```
let first_space_flight = 1961;

let n = 12.345;
```

**Тип данных строка**

Строка (`string`) в JavaScript должна быть заключена в кавычки.

```
let message = "Это строка";
```

В JavaScript существует три типа кавычек.

1. Двойные кавычки: `"МГТУ им. Н.Э. Баумана"`.
2. Одинарные кавычки: `'МГТУ им. Н.Э. Баумана'`.
3. Обратные кавычки: `` `МГТУ им. Н.Э. Баумана` ``.

В javascript нет разницы между одинарными и двойными кавычками, однако обратные кавычки имеют расширенную функциональность.
Они позволяют встраивать выражения в строку, заключая их в `${…}`. Например:

```
let name = "Никола́й Эрне́стович Ба́уман"

// Вставим переменную
alert(`Здравствуйте, ${name}!`);   // Здравствуйте, Никола́й Эрне́стович
Ба́уман!

// Вставим выражение
alert(`результат: ${8+2}`);       // результат: 10
```

Выражение внутри `${…}` вычисляется, и его результат становится частью строки. Обратите внимание, что это можно делать только в обратных кавычках. Другие кавычки не имеют такой функциональности встраивания!

**Конкатенация**

Познакомимся еще с одним понятием.
Конкатенация означает объединение двух строк. В JavaScript конкатенация строк выполняется с использованием оператора `+`.

Например:

```
var str1 = "Hello";
var str2 = "World";
var result = str1 + " " + str2; // результат: "Hello World"
```

В примере оператор `+` используется для объединения (конкатенации) двух строк str1 и str2, а также добавления пробела между ними.

**Функции**

**Функция** — это именованный блок кода, предназначенный для повторного выполнения какой-либо задачи.Они могут принимать параметры и возвращать значение.
Простейшая функция выглядит так:

```
function sayHello(name) {
  result = "Привет, " + name
  alert(result);

  return result;
}

sayHello("Мир");
```

Опишем устройство функции в объеме, необходимом для дальнейшего изложения. Вначале идёт ключевое слово `function`, после него `имя функции`, затем список `параметров` в круглых скобках через запятую (в вышеприведённом примере

функцией принимается один параметр name) и, наконец, код функции, также называемый «телом функции», внутри фигурных скобок.
Наша новая функция может быть вызвана по своему имени: `sayHello("Мир")`.

Кроме того, из функции можно возвращать значения с помощью оператора `return`

**Условия и циклы**

JavaScript поддерживает условные операторы и циклы для управления потоком выполнения кода.

```
if (age > 18) {
  alert("Вы совершеннолетний!");
} else {
  alert("Вы ещё не совершеннолетний!");
}
```

Опишем этот код подробнее.
В представленном выше кодовом блоке проверяется условие `age > 18`. Если переменная age больше 18, то выполняется блок кода внутри фигурных скобок после if, который выводит сообщение "Вы совершеннолетний!" с помощью функции alert(). Если условие не выполняется (то есть переменная age меньше или равна 18), то выполняется блок кода после else, который выводит сообщение "Вы ещё не совершеннолетний!" с помощью функции alert().

```
for (let i = 0; i < 5; i++) {
  console.log(i);
}
```

А в данном блоке кода находится цикл. Этот цикл итерируется от 0 до 4 (пять итераций) с помощью переменной i, которая начинается с 0 и увеличивается на 1 на каждой итерации. На каждой итерации внутри цикла вызывается функция console.log(), которая выводит значение переменной i в консоль браузера.

**Объектная модель документа**

Совсем скоро мы научимся использовать javascript на практике для "оживления" страницы. Но для этого нужно познакомиться с одним очень важным понятием. Оно называется объектная модель документа.
Объектная модель документа (Document Object Model, DOM) в JavaScript представляет собой иерархическую структуру, которая представляет веб-страницу как дерево объектов. Каждый элемент HTML на странице представлен как объект в этой модели, и вы можете взаимодействовать с этими объектами с помощью JavaScript для изменения содержимого, стилей и поведения страницы.

Объектная модель документа изображена на рисунке 27.

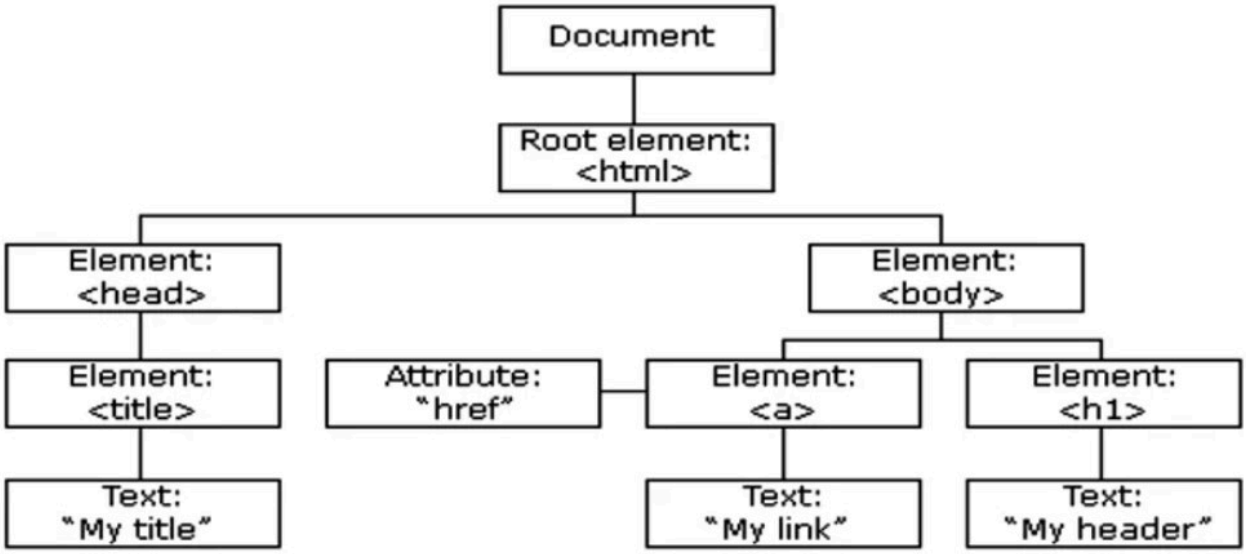



Рис. 27. Объектная модель документа

Приведем пример.
На странице contacts.html у нас HTML-код:

```
<main>
   <div class="contacts__wrapper">
        <div class="contacts__header">
            <h1>Связаться с нами</h1>
        </div>
        <form class="contacts__form" action="#" method="#">
            <label class="contacts__form-label" for="username">Как вас
зовут?</label>
            <input class="contacts__form-input" type="text" id="username"
name="username"><br><br>
            <label class="contacts__form-label" for="message">Сообщение
</label>
            <textarea class="contacts__form-textarea"  rows="5"
name="message" placeholder="Расскажите обо всём, что для вас важно"
id="message"></textarea>
            <br><br>
            <input class="contacts__form-submit" type="submit"
value="Отправить">
          </form>
    </div>
  </main>
```

С помощью JavaScript мы можем получить доступ к элементам на странице и изменить их. Попробуем это сделать.

В файле scripts.js закомментируйте код предыдущего примера и добавьте следующий код:

```
// let message = "Привет, мир!";
// alert(message);

// Получаем элемент заголовка по его тэгу
let headingElement = document.querySelector('h1');

// Меняем текст заголовка
headingElement.textContent = 'Вы хотите связаться с нами? Заполните форму ниже!';

// Меняем цвет текста заголовка на красный
headingElement.style.color = 'red';
```

Сохраните файл и откройте страницу contacts.html в браузере (Рис.28).

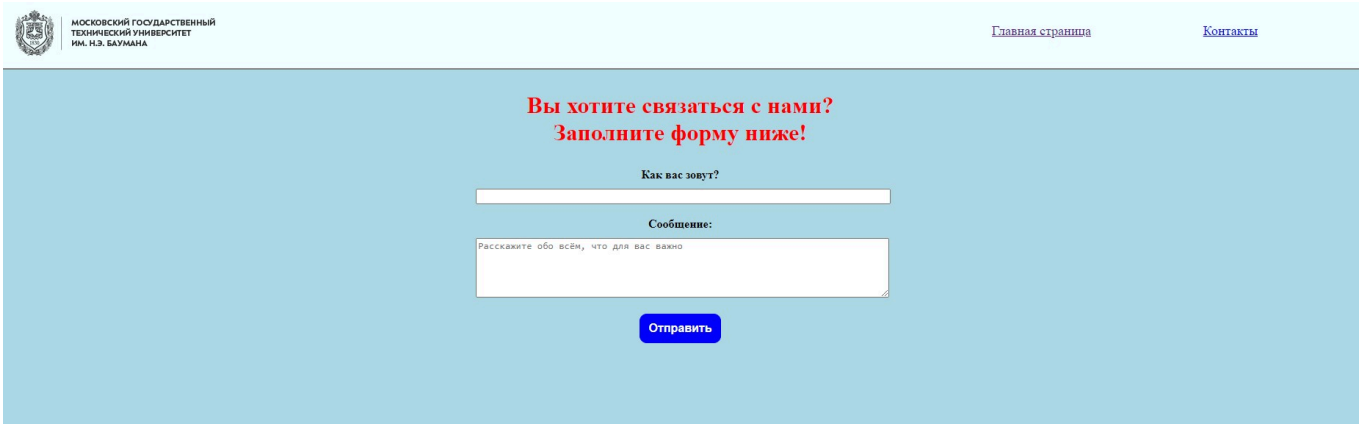



Рис. 28. Мы изменили содержимое веб страницы с помощью языка JavaScript.

В этом примере document - это объект, который представляет всю веб-страницу, а querySelector('h1') - метод для получения элемента по его тэгу. Затем мы используем свойство textContent объекта элемента, чтобы изменить текст.
Когда мы используем headingElement.style.color = 'red'; , мы обращаемся к свойству style элемента headerElement, которое представляет собой место, содержащее все стили этого элемента. Затем мы устанавливаем значение свойства color этого элемента равным 'red', что приводит к изменению цвета текста заголовка на красный.

Это действие эквивалентно изменению цвета с помощью CSS.

DOM также позволяет добавлять новые элементы на страницу, удалять существующие, обрабатывать события и многое другое. Вся эта работа выполняется с помощью JavaScript, который взаимодействует с объектами DOM.

Объектная модель документа является основой для динамического обновления веб-страниц и создания интерактивных пользовательских интерфейсов.

**События**

JavaScript может реагировать на **события**, происходящие на странице: щелчки мыши, ввод текста, загрузку документа и т.д.

Приведем небольшой список типов событий, которые JavaScript может обрабатывать.

1. События мыши:
   - click - клик мышью
   - mouseover - наведение мыши
   - mouseout - уведение мыши
   - mousedown - нажатие кнопки мыши
   - mouseup - отпускание кнопки мыши
2. События клавиатуры:
   - keydown - нажатие клавиши
   - keyup - отпускание клавиши
   - keypress - нажатие и удержание клавиши
3. События формы:
   - submit - отправка формы
   - change - изменение значения элемента формы
   - focus - фокусировка на элементе формы
   - blur - потеря фокуса элементом формы
4. События документа:
   - DOMContentLoaded - загрузка DOM дерева документа
   - load - полная загрузка документа (включая ресурсы)
   - scroll - прокрутка документа
5. События окна браузера:
   - resize - изменение размеров окна браузера
   - unload - закрытие окна или переход на другую страницу
6. Другие события:
   - error - возникновение ошибки
   - offline и online - изменение статуса сети

Продемонстрируем обработку события на примере события "Клик"

Допишем немного javascript в файл script.js:

```
// Получаем элемент заголовка по его id
let headingElement = document.querySelector('h1');

// Меняем текст заголовка
headingElement.textContent = 'Вы хотите связаться с нами? Заполните форму
ниже!';

// Меняем цвет текста заголовка на красный
```

```
headingElement.style.color = 'red';

// Получаем элемент заголовка по его id
let formButton = document.querySelector('.contacts__form-submit');


// НИЖЕ НОВЫЙ КОД


// функция генерирует случайный цвет в формате hex-кода
function getRandomColor() {
    let letters = '0123456789ABCDEF';
    let color = '#';
    for (let i = 0; i < 6; i++) {
      color += letters[Math.floor(Math.random() * 16)];
    }
    return color;
  }


// Получаем элемент main по его тэгу
let mainElement = document.querySelector('main');

// Навешиваем на кнопку обработчик клика
formButton.onclick = function(event) {
    // Отменяем действие по умолчанию
    event.preventDefault();
    // Вызываем функцию getRandomColor, чтобы давать элементам случайные
цвета
    // при нажатии кнопки
    headingElement.style.color = getRandomColor();

    // элементу main меняем цвет фона при нажатии
    mainElement.style.backgroundColor = getRandomColor();
}
```

Мы создали новую функцию getRandomColor. Ее назначение - генерировать случайный цвет в формате hex-кода.

Далее код:

1. Получает элемент `main` с помощью метода `document.querySelector('main')` и сохраняет его в переменную `mainElement`.
2. Навешивает обработчик события клика на кнопку `formButton`. Когда кнопка будет нажата, будет выполнен следующий код.
3. Внутри обработчика события:

- Отменяется действие по умолчанию при помощи `event.preventDefault()`, чтобы предотвратить перезагрузку страницы.
- Вызывается функция `getRandomColor()` для получения случайного цвета.
- Цвет текста заголовка (`headingElement`) изменяется на случайный цвет при помощи `headingElement.style.color = getRandomColor()`.
- Цвет фона элемента main меняется на случайный цвет при помощи `mainElement.style.backgroundColor = getRandomColor()`.

Таким образом, при каждом нажатии на кнопку (обработка события клика с помощью обработчика `onclick`) будет изменяться цвет текста заголовка и цвет фона элемента `main` на случайные цвета, полученные с помощью функции `getRandomColor()`.

Сохраним файл и откроем страницу contacts.html в браузере (Рис.29)

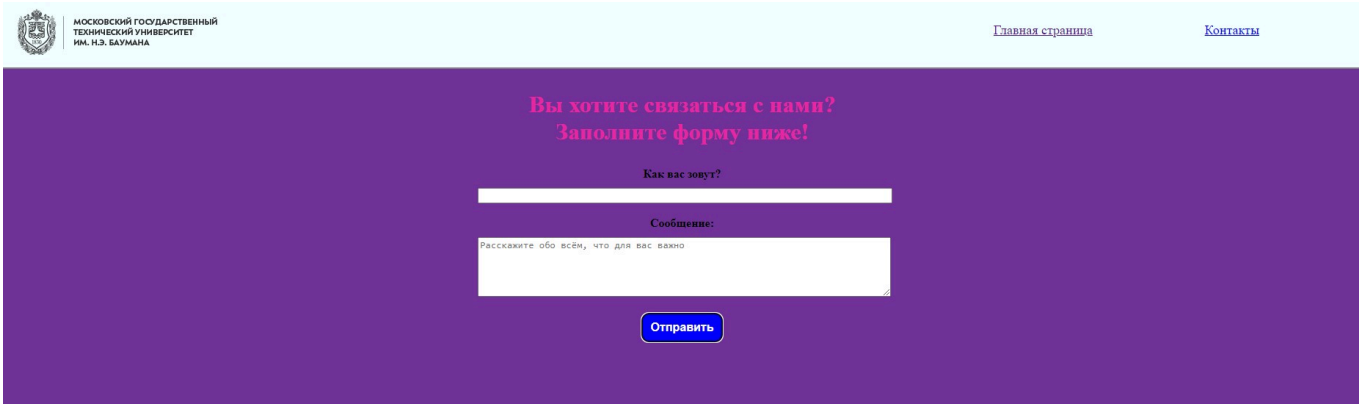



Рис. 29. Страница contacts.html с измененными цветами.

Теперь при каждом нажатии на кнопку цвета на странице будут меняться.

Обратим внимание, что можно назначить обработчик события прямо в разметке используя атрибуты событий. Значением атрибута события должен быть скрипт на языке javascript.

Приведем пример:

```
<img src="" onerror="alert('Произошла ошибка загрузки изображения')">
```

В данном разделе мы познакомились с основами javascript.

Самое время перейти к рассмотрению первой веб-уязвимости.